WRITEUP · ÖFFENTLICH · TEIL 2

LibreLinkUp Reverse Engineering — Teil 2

Vom entschlüsselten Binary zum .so-freien Client (lipy)

Dies ist die Fortsetzung des öffentlichen Teils 1 („LibreLinkUp Application-Level-Encryption — ein Reverse-Engineering-Writeup"), der das Gesamtsystem, die Schutzschichten, den Live-Hook und das White-Box-Orakel behandelt hat. Dieser zweite Teil bündelt das darauffolgende Ingenieurs-Ergebnis: Die Trennung der zwei Krypto-Welten der App, die drei Wege an die entschlüsselte native Bibliothek, den vollständigen Nachbau der Kryptografie in reinem Python, die Wartung über App-Updates hinweg, zwei Fault-Analysen sowie eine Analyse des privaten Signierwerks, die zeigt, warum der RSA-Schlüssel den untersuchten Live-Extraktionen standhielt. Am Ende steht ein eigenständiger Client, der den eigenen Glukosewert allein aus E-Mail-Adresse und Passwort ausliest.

Autor: IamDiesel · **Stand:** November 2026 · **Version:** 1.2 · **Lizenz:** CC BY 4.0

Testumgebung: LibreLinkUp 5.1.1 und 5.2.0 · Google Pixel 9 Pro, Android 17 (ungerootet, kein Root) · **Analysezeitraum:** August–Oktober 2026 · **KI-Einsatz:** Analyse, Software und Bericht

Status: Fortsetzung von Teil 1. Das Interoperabilitäts-Ziel ist erreicht, durchgehend auf einem ungerooteten Gerät und ausschließlich an eigenen Daten. Der Fokus liegt auf der technischen Umsetzung. Auch dieser Teil ist bewusst ein Erkenntnis-Bericht und keine Schritt-für-Schritt-Anleitung zum Nachbau.

Medizinischer Haftungsausschluss: Dies ist ein technischer Sicherheits- und Reverse-Engineering-Bericht und kein medizinisches Dokument. Die beschriebenen Verfahren und sämtliche damit gewonnenen Glukosewerte (CGM-Daten) dürfen unter keinen Umständen für medizinische Entscheidungen herangezogen werden. Das gilt insbesondere für die Dosierung von Insulin oder andere Therapieanpassungen. Selbst erstellte Auslese-Wege können fehlerhafte, verzögerte, veraltete oder unvollständige Werte liefern. Für Therapieentscheidungen sind ausschließlich die vom Hersteller zugelassenen, geprüften Anzeigegeräte und Apps sowie ärztlicher Rat maßgeblich.

Rahmen und Ethik: Alle Arbeiten fanden ausschließlich an eigenen Daten und auf eigenen, ungerooteten Geräten ohne Root-Zugriff statt. Es erfolgt keinerlei Weitergabe von Schlüsselmaterial, Klartexten oder personenbezogenen Daten. Dieser öffentliche Teil enthält bewusst keine Build-Offsets, Adressen, Schlüssel, Krypto-Konstanten, Skalare, Tabellen-Bytes, internen Klassen- oder Methodennamen oder Smali-Patch-Rezepte. Diese Details verbleiben im nicht-öffentlichen Arbeitsarchiv. Zweck der Veröffentlichung ist der Wissensaufbau in den Bereichen Reverse Engineering, Kryptografie und IT-Sicherheit.

Inhalt

1. Von der Analyse zur Umsetzung ... 3
2. Die zwei Krypto-Welten und die falsche Tür ... 4
3. Drei Wege zur entschlüsselten .so ... 5
4. Das Offline-Entpacken der DexGuard-Hülle ... 6
5. Vier Ebenen, dieselbe Box ... 9
6. Der reine Python-ARM64-Interpreter ... 10
7. Funktions-Identifikation und ABI per Beobachtung ... 11
8. Der NEON-Durchbruch ... 12
9. Die White-Box ausführbar reproduziert ... 13
10. Der entscheidende Hebel: SK2 ist Standard-AES-256-GCM ... 14
11. Der Snapshot statt der .so ... 16
12. Die .so-freie Provisionierung und der Client ... 17
13. Performance-Engineering ... 19
14. Register-genaue NEON-Stubs: Der beobachtbare Signierpfad wird praxistauglich ... 21
15. Offset-Migration und Wartung ... 23
16. Produktionsreife und Validierung ... 24
17. Fault-Analyse A: DFA gegen die AES-White-Box ... 26
18. Fault-Analyse B: RSA-PSS und warum non-CRT den Bellcore-Angriff aushebelt ... 28
19. Das private Signierwerk: Warum die untersuchten Live-Extraktionen den geheimen Exponenten nicht preisgeben ... 30
20. Was die Umsetzung zeigt: Die Sicherheits-Lehre ... 33
21. Ethik und Geltungsbereich ... 34
Glossar ... 35
Referenzen ... 37

1. Von der Analyse zur Umsetzung

Der Kern dieses zweiten Teils ist der Schritt vom Nachnutzen des Hersteller-Binaries zum bibliotheksfreien Nachbau. Teil 1 [1] endete zwar bereits bei einem eigenständigen Client, der den Glukosewert nur mit E-Mail-Adresse und Passwort abrief. Er lud hierfür aber noch die native Krypto-Bibliothek (.so) des Herstellers und nutzte sie als Orakel (Code Lifting). Teil 2 löst nun auch diese letzte Abhängigkeit. Große Teile der White-Box-Kryptografie und die zeitkritischen Schleifen setzen wir direkt in Python um. Den übrigen Kontrollfluss übernimmt ein eigener, bitgenauer Interpreter, während ein kompakter Snapshot die statischen Tabellen bereithält. Damit entfällt die .so-Datei vollständig und im Betrieb wird kein nativer Maschinencode mehr ausgeführt. Das Ziel bleibt der eigene Wert auf der eigenen Uhr. Neu ist das Ergebnis: Ein reiner Python-Client, der ohne Smartphone, App und Android-Umgebung auf beliebiger Hardware läuft. Wie schon Teil 1 dient auch dieser Beitrag dem Erkenntnisgewinn: Das technische Verständnis steht im Vordergrund, nicht ein Rezept zum Nachbau.

> Teil 1 hat gezeigt, wie die App ihre Daten schützt und wie man sie von außen beobachtet. Dieses Kapitel gibt den Gesamtüberblick darüber, wie aus dieser Beobachtung ein unabhängiger Python-Client entsteht, der weder ein Smartphone noch die binäre Bibliothek des Herstellers benötigt.

Zur Erinnerung: Die App sichert ihre Nutzlast zusätzlich zu TLS mit einer eigenen Application-Level Encryption (ALE) ab. Zunächst gelangte man auf zwei Wegen an den Klartext: Entweder per Hardware-Breakpoint-Hook direkt am Gerät oder über die native White-Box-Bibliothek als Orakel im eigenen Prozess.

Konkret beschreibt dieser zweite Teil den Weg vom noch geräte- und bibliotheksgebundenen Orakel aus Teil 1 zum vollständig `.so`-freien Python-Client `lipy`:

- **Vorarbeit (§2 bis §4):** Wir trennen zunächst die zwei Krypto-Welten der App und schließen die „falsche Tür“ des lokalen Speichers (§2). Danach gewinnen wir die verschlüsselte native Bibliothek, zuletzt komplett offline direkt aus der APK-Datei (§3 und §4).
- **Bit-exakter Python-Nachbau der Engine (§5 bis §9):** Wir bauen die Laufzeitumgebung samt Objekt-Prägung (*Replay-Mint*) in einem reinen Python-ARM64-Interpreter nach (§5 bis §7), ergänzen die sicherheitskritischen NEON-Vektorbefehle (§8) und legen das Tabellennetz der White-Box bis auf seine elf Blatt-Primitive frei (§9).
- **Reduktion, Snapshot und Beschleunigung (§10 bis §14):** Wir optimieren das System schrittweise vom minutenlangen Emulator-Lauf auf Sekundenbruchteile im Alltag: Die kryptografische Reduktion überführt den Dauerbetrieb in Standard-AES-256-GCM [7] und reduziert die Schlüsselableitung auf wenige Blockaufrufe (§10). Ein vorgewärmter Speicher-Snapshot ersetzt die `.so`-Datei samt Initialisierung (§11). Gezieltes Performance-Engineering beschleunigt den einmaligen Live-Handshake (§12) von rund 320 s über etwa 120 s (§13) bis auf rund 6,4 s reine Signaturzeit (§14).
- **Wartung und Produkt-Architektur (§15 und §16):** Wir zeigen, warum die meisten App-Updates keinen Eingriff an der Engine erfordern (§15) und gießen den Client in eine produktive Zwei-Zonen-Architektur mit strikter IP-Trennung und Offline-Validierung (§16).
- **Kryptanalyse und Sicherheits-Lehre (§17 bis §21):** Weil die gesamte Engine nun in Python vorliegt, eröffnet sich die einmalige Möglichkeit, die White-Box abseits aller Smartphone-Schutzmechanismen wie unter einem Mikroskop zu untersuchen: Wir prüfen in zwei gezielten Fault-Analysen, ob sie ihre eingebetteten AES- und RSA-Schlüssel preisgibt (§17 und §18), öffnen

anschließend das private Signierwerk und zeigen, warum der geheime Exponent den untersuchten Live-Extraktionen standhielt (§19) und ziehen daraus die zentrale Architektur-Lehre für moderne App-Sicherheit (§20 und §21).

Das Grundmodell in drei Sätzen (als Kompass für alle folgenden Kapitel): Die White-Box enthält zwei statische, in jeder App-Installation identische Objekte: Einen privaten Signaturschlüssel (`data1`, RSA-2048-PSS) und einen statischen Schlüssel-Verschlüsselungsschlüssel (`data2`, AES-256). Einmal pro Registrierung sendet der Client einen mit `data2` verschlüsselten und mit `data1` signierten Handshake an den Server und erhält dafür ein mit `data2` eingewickeltes Schlüsselpaket (`kp`) zurück. Aus diesem Paket entpackt der Client den 32 Byte langen Sitzungsschlüssel `SK2` und entschlüsselt fortan jede Glukose-Antwort direkt mit `SK2`.

Um dieses Modell sauber umzusetzen, muss man zunächst eine verlockende Fehlspur ausschließen, die bei der Analyse moderner Apps fast immer zuerst ins Auge fällt: Den lokalen Speicher.

2. Die zwei Krypto-Welten und die falsche Tür

> Wer eine App analysiert, sucht Schlüssel oft zuerst in den lokalen Datenbanken auf dem Gerät. Dieses Kapitel zeigt, warum das Knacken des lokalen Speichers hier in eine Sackgasse führt und warum für die Glukosedaten ausschließlich die Transport-Verschlüsselung (Data-in-Transit) relevant ist.

Die App verschlüsselt Daten in zwei voneinander völlig unabhängigen Welten. Das Projekt verlor in einer frühen Phase viel Zeit an der Falschen davon:

1. **Data-in-Transit (der echte Weg):** Das ist der API-Verkehr über das Netzwerk, also die ALE-Schicht innerhalb von TLS, geschützt mit AES-256-GCM unter dem Sitzungsschlüssel `SK2`. Genau hier wird der aktuelle Glukosewert übertragen und diesen Pfad liest `lipy`.
2. **Data-at-Rest (die falsche Tür):** Das ist der lokale Speicher auf dem Smartphone. Er besteht aus einem Jetpack-DataStore (`Tink`-Keyset unter AES-256-GCM) und einem lokalen Zustandsspeicher (`MMKV` unter AES-128-CFB). Der oberste Master-Schlüssel dieser lokalen Kette wird durch den Titan-M-Sicherheitschip des Telefons geschützt. *(Aus der 128-Bit-MMKV-Schicht stammte übrigens auch die lange kursierende Fehlannahme, die API selbst nutze „AES-GCM-128“.)*

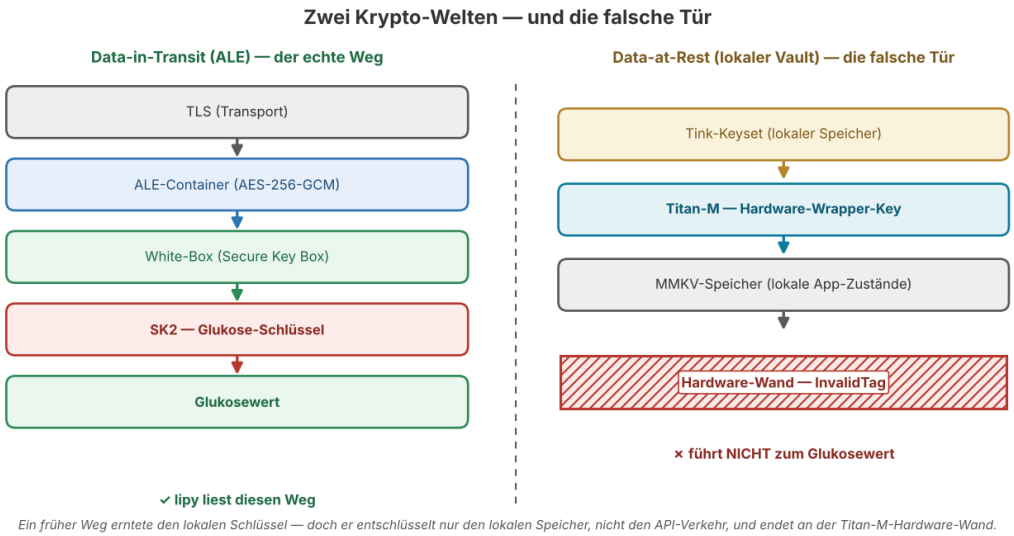


Abbildung 1: Zwei Krypto-Welten: Data-in-Transit (links) trägt über die White-Box den Glukose-Schlüssel und damit den Wert. Das ist der echte Weg von lipy. Data-at-Rest (rechts) ist der lokale Vault. Er endet an der Titan-M-Hardware-Wand und führt nicht zum Glukosewert.

Der lokale Vault ließ sich zwar zur Laufzeit öffnen: Eine Sonde an der Java-Krypto-Grenze griff den lokalen Schlüssel ab, womit der lokale Speicher vollständig entschlüsselt werden konnte. Dieser vermeintliche Durchbruch entpuppte sich jedoch aus drei Gründen als falsche Tür:

- **Erstens (falsche Domäne):** Der lokale Schlüssel entschlüsselt ausschließlich den lokalen Speicher (etwa gespeicherte Login-Tokens), aber nicht den Netzwerkverkehr. Jeder Versuch, damit API-Pakete zu öffnen, bricht sofort mit einem `InvalidTag`-Fehler ab.
- **Zweitens (nur ein Chiffretext-Cache):** Das im lokalen `MMKV`-Speicher abgelegte API-Material (`secureApiData`) ist kein fertiger Klartext-Schlüssel, sondern eine byte-identische Kopie des vom Server gelieferten Schlüsselpakets (`kp`). Es handelt sich also selbst um einen White-Box-Chiffretext unter `data2`, den nur die White-Box entpacken kann.
- **Drittens (die Hardware-Wand):** Der oberste Wrapper-Schlüssel des Android-Keystores verlässt den Titan-M-Sicherheitschip nie im Klartext. Eine reine Offline-Entschlüsselung des Hardware-Tresors ohne laufende App ist damit unmöglich und endet ebenfalls in `InvalidTag`.

> **Lehre und Abgrenzung:** Alle Wege führen zwingend zurück zur White-Box der Transit-Welt. Der hardware-gestützte Titan-M-Keystore wurde an keiner Stelle gebrochen und wird für den Zugriff auf die Glukosedaten auch gar nicht berührt. Damit ist das eigentliche Ziel klar eingegrenzt: Wir benötigen die native White-Box-Bibliothek im Klartext.

3. Drei Wege zur entschlüsselten .so

> Die Krypto-Bibliothek liegt nicht offen in der Installationsdatei (APK), sondern ist dort verschlüsselt verpackt. Bevor man ihren Code lesen oder in Python nachbauen kann, muss man das entschlüsselte ELF-Binary gewinnen (ELF ist das ausführbare Binärformat unter Linux und Android). Dieses Kapitel stellt die drei entwickelten Extraktionswege vor.

Die gesamte Transit-Kryptografie steckt in einer rund 13 MB großen nativen Bibliothek, die im APK mit maximaler Entropie (8,00 Bit/Byte) verschlüsselt ist und erst beim App-Start entpackt wird. Um ihr entschlüsseltes Abbild zu erhalten, entstanden im Projektverlauf drei Verfahren mit zunehmender Unabhängigkeit vom Smartphone.

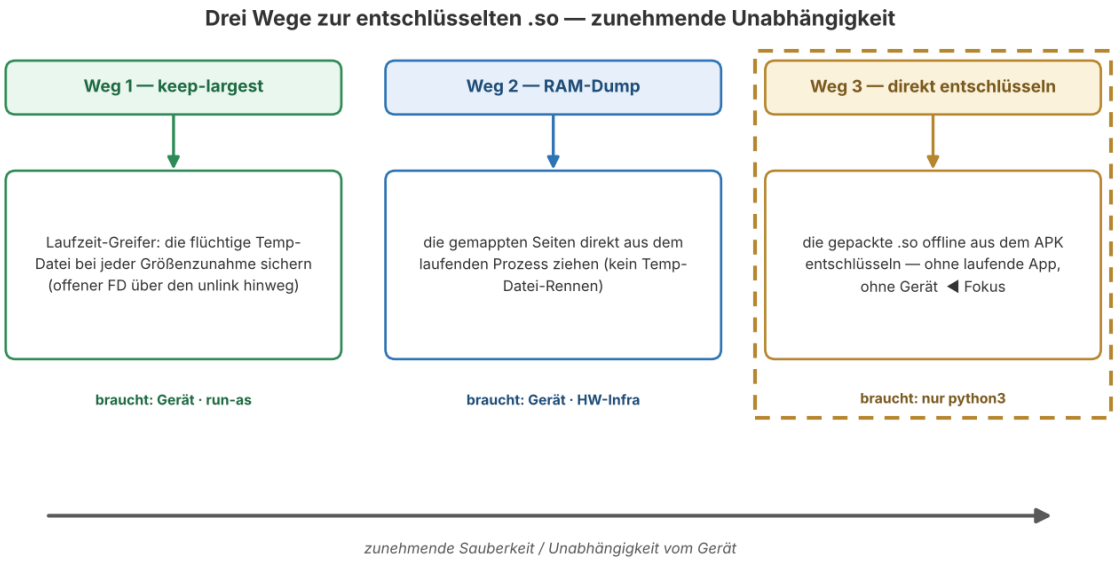


Abbildung 2: Drei Wege zur entschlüsselten .so: Der keep-largest-Laufzeit-Greifer, der RAM-Dump aus dem laufenden Prozess und als sauberste, gerätefreie Variante das direkte Offline-Entschlüsseln der gepackten Datei.

- **Weg 1:** `keep-largest` **(Laufzeit-Greifer beim Kaltstart).** Der Packer schreibt die entschlüsselte Bibliothek beim Start der App in eine flüchtige temporäre Datei, lädt sie in den Speicher und löscht den Dateinamen (`unlink`) sofort wieder. Solange ein Kopierprozess aber einen offenen Datei-Deskriptor auf diese Datei hält, bleibt ihr Inhalt im Dateisystem bestehen. Eine einfache Schleife, die die Datei bei jeder Größenzunahme neu sichert, greift so die vollständige entschlüsselte Bibliothek ab. Voraussetzung ist ein gekoppeltes Gerät mit `run-as`-Zugriff (ohne Root).
- **Weg 2:** `RAM-Dump` **(Abgriff aus dem laufenden Prozess).** Um das zeitkritische Wettrennen mit der temporären Datei beim Kaltstart zu vermeiden, liest man die bereits fertig gemappte Bibliothek direkt aus dem Arbeitsspeicher des laufenden Prozesses aus. Das ist deutlich robuster, erfordert aber weiterhin ein physisches Gerät, auf dem die App läuft.
- **Weg 3:** `Direkt entschlüsseln` **(statischer Offline-Entpacker).** Die gepackte Datei wird direkt auf dem PC aus dem APK entschlüsselt. Dafür wird weder eine laufende App noch ein Smartphone benötigt. Ein reines Python-Skript genügt.

Warum Weg 3 das Fundament bildet: Die ersten beiden Wege waren wichtige Meilensteine, binden den Analysten aber an ein Telefon. Erst das direkte Offline-Entpacken macht die gesamte Kette von der ersten Sekunde an gerätefrei. Wie man den kommerziellen Packer dafür überlistet, ohne bei jedem App-Update von vorn anzufangen, zeigt Kapitel 4.

4. Das Offline-Entpacken der DexGuard-Hülle

> Kommerzielle App-Packer ändern ihre Schlüssel und Verschleierungsparameter bei jedem neuen Build. Dieses Kapitel erklärt die interne Mechanik des Packers und zeigt einen Kalibrierungs-Kniff, mit dem sich auch künftige App-Versionen vollautomatisch auf dem PC entpacken lassen.

Der eingesetzte Schutzmechanismus (DexGuard) liefert mehrere native Bibliotheken verschlüsselt im APK aus. Entgegen einer häufigen Annahme nutzt er dafür keinen eigenen nativen System-Linker: Zur Laufzeit entschlüsseln Java-Loader-Methoden den Datenstrom über einen vorgeschalteten Filter-Eingabestrom

(`FilterInputStream`) in eine temporäre Datei und übergeben diese anschließend ganz regulär per `System.load` an den Betriebssystem-Linker, bevor die temporäre Datei sofort wieder gelöscht wird. Genau dieser Java-Ablauf lässt sich offline in Python nachbilden.

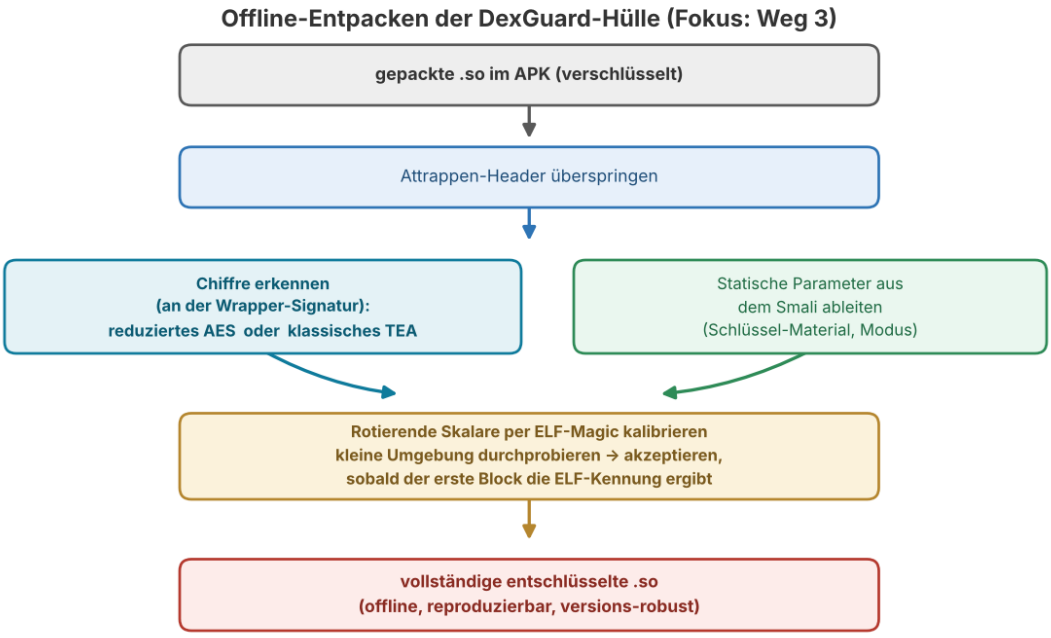


Abbildung 3: Offline-Entpacken: Attrappen-Header überspringen, die Chiffre an der Wrapper-Signatur erkennen (reduziertes AES oder klassisches TEA), die statischen Parameter aus dem Smali ableiten und die rotierenden Skalare per ELF-Magic kalibrieren, bis die vollständige entschlüsselte .so offline vorliegt.

4.1 Die Mechanik des Packers

- **Attrappen-Header:** Die ersten 269 Byte der gepackten Datei sind ein konstruierter Dummy-Header, der beim Lesen per skip übersprungen wird. Da er mit den echten \x7f ELF-Magic-Bytes beginnt, aber nur leere Program-Header (filesz = 0) ohne Sektionen enthält, täuscht ein naiver Blick auf den Dateianfang eine unverschlüsselte Bibliothek vor. Erst hinter diesem Vorspann beginnt der eigentliche Chiffretext.
- **Zwei reduzierte, nicht-standardkonforme Chiffren:** Ein Teil der Bibliotheken (darunter die White-Box) ist mit einem **verkürzten AES-128** im ECB-Modus verschlüsselt. Es durchläuft nur drei statt zehn Runden, nutzt einen bitmaskierten Rohschlüssel und verwendet eine pro Build frei parametrisierte `ShiftRows`-Permutation. Ein anderer Teil der Bibliotheken ist mit **klassischem TEA** (Tiny Encryption Algorithm) im CBC-Modus und fünf Runden gepackt. Weil beide Chiffren bewusst vom Standard abweichen, können gewöhnliche Krypto-Bibliotheken sie nicht entschlüsseln.
- **Statische Schlüssel und rotierende Opaque Predicates:** Das primäre Schlüsselmaterial liegt als statisches Array in den Java-Loader-Klassen im Smali-Code. Beim Versionswechsel rotieren jedoch nicht nur dieser Rohschlüssel und der Bibliotheksname, sondern noch zwei weitere Größen: Zum einen das 16-Bit-Auswahlmuster der Schlüsselableitung (das bitweise festlegt, welche der 16 Rohschlüssel-Bytes mit einem über Versionen stabilen Masken-Byte ge-XORt werden) und zum anderen einzelne Steuer-Skalare wie die Rundenzahl oder die ShiftRows-Steuerzahl. Letztere werden zur Laufzeit aus Opaque Predicates berechnet. Das sind Android-

API-Aufrufe, die auf einem echten Gerät zwar immer denselben konstanten Wert liefern, statisch im Code aber verschleiert sind und bei jedem Build neu gewürfelt werden.

4.2 Der robuste Extraktor: Kalibrierung per ELF-Magic

Würde man versuchen, die Opaque Predicates und Ableitungsmasken jeder neuen App-Version im Smali-Code manuell nachzurechnen, wäre der Entpacker fragil. Der entscheidende Kniff besteht darin, die entschlüsselte Nutzlast selbst als Orakel zu nutzen: Das Python-Werkzeug liest die Loader-Methode aus dem Smali-Code, erkennt die Chiffre automatisch an der Methodensignatur, extrahiert das statische Rohschlüssel-Array und kalibriert die rotierenden Parameter in zwei Stufen direkt gegen den ersten 16-Byte-Chiffretextblock. Ein Schnellpfad prüft zunächst die bekannte Standard-Bitmaske zusammen mit einer winzigen Zahlen-Umgebung für die Steuer-Skalare. Greift diese bei einer anderen App-Version nicht, durchläuft ein Fallback automatisch auch den geschlossenen 16-Bit-Raum des Masken-Bitmusters.

Sobald der erste entschlüsselte 16-Byte-Block mit der eindeutigen, 8 Byte langen ELF64/AArch64-Kennung (7F 45 4C 46 02 01 01 00) beginnt, sind die Parameter-Kandidaten stark eingegrenzt. Für acht feste Bytes liegt die Wahrscheinlichkeit eines reinen Zufallstreffers bei etwa 2^{-64}, sofern die geprüften Kandidaten die Bytes hinreichend gleichverteilt erzeugen. Die Magic ist dabei nur ein Format-Indikator, kein vollständiger Korrektheitsbeweis. Abgesichert wird der Treffer daher zusätzlich über weitere ELF-Struktur-Invarianten (Header-Felder, plausible Program-Header mit realer Dateigröße und gültige Segmentgrenzen). Dies verhindert zugleich auf einen Attrappen-Header hereinzufallen. Auf diese Weise ließen sich beide untersuchten App-Versionen (5.1.1 und 5.2.0) direkt aus ihren APK-Archiven vollautomatisch entpacken: Trotz komplett rotierter Bibliotheksnamen, Rohschlüssel, Masken-Bitmuster und Steuer-Skalare liefern beide Builds bis auf das letzte Byte exakt dieselbe entschlüsselte White-Box-Bibliothek ($\Delta = 0$, identischer SHA-256-Hash wie der Laufzeit-Dump vom Gerät).

Diese Kalibrierung setzt allerdings voraus, dass die entschlüsselte Nutzlast tatsächlich ein Standard-ELF ist und mit dessen Kennung beginnt. Würde der Hersteller die Payload zusätzlich transformieren, doppelt verpacken oder in einen anderen Container legen, verlöre das ELF-Magic-Orakel seinen Ankerpunkt. Robuster wird das Verfahren, indem man das Orakel verallgemeinert, statt allein der Magic weitere Struktur-Invarianten des Formats als Plausibilitätsmaß prüft und als Rückfallebene die rotierenden Skalare doch exakt aus dem Smali-Code nachrechnet. Die Kombination aus beidem ist am widerstandsfähigsten.

Mit dem entschlüsselten Binary auf der Festplatte stehen wir vor der zentralen Architekturfrage des Projekts: Wie führen wir den darin enthaltenen ARM64-Maschinencode aus, ohne ein Smartphone zu benutzen und ohne am Ende die `.so`-Datei selbst mitzuliefern?

5. Vier Ebenen, dieselbe Box

> Zwischen dem Ausführen einer nativen Bibliothek auf dem Handy und einer reinen Python-Neuprogrammierung liegen mehrere Abstufungen. Dieses Kapitel ordnet die vier möglichen Ausführungsebenen ein und erklärt, was „`.so`-frei“ in `lipy` technisch genau bedeutet.

Um die White-Box zum Rechnen zu bringen, wurden im Projekt vier aufeinander aufbauende Ausführungsebenen umgesetzt. Es handelt sich dabei nicht um ungefähre Annäherungen, sondern um vier verschiedene Arten, denselben Original-Code auszuführen.

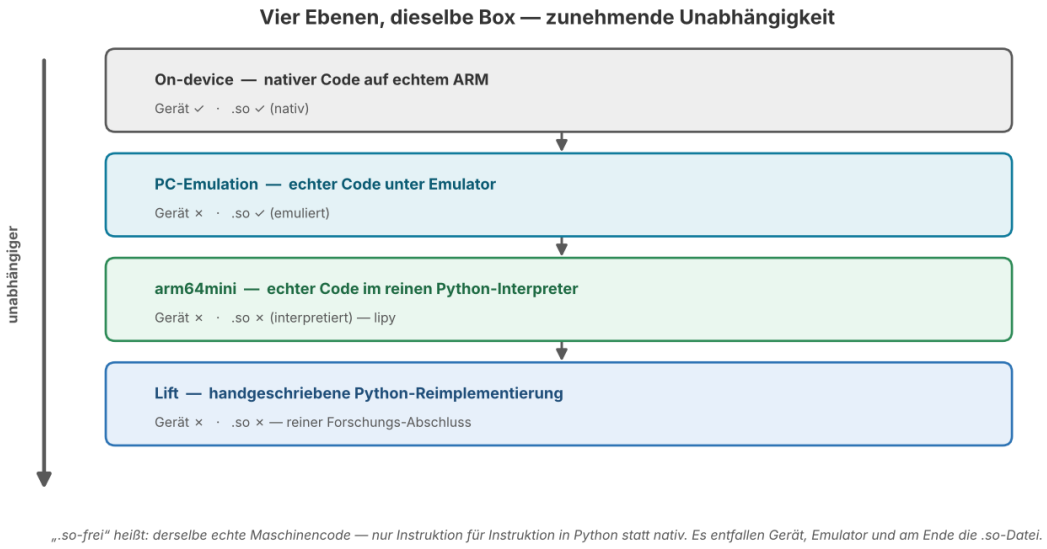


Abbildung 4: Vier Ebenen, dieselbe Box: Vom nativen On-device-Lauf über die PC-Emulation zum reinen Python-Interpreter (lipy) und schließlich zur handgeschriebenen Reimplementierung. Mit jeder Ebene entfällt eine Abhängigkeit.

1. **On-device (nativ auf echtem ARM):** Die originale `.so`-Datei läuft direkt auf dem Smartphone, entweder beobachtet in der Original-App oder eingebunden in eine eigene Android-App (`GlucoBridge`).
2. **PC-Emulation:** Der echte Maschinencode läuft unter einem C-basierten CPU-Emulator (`unicorn`) auf dem PC. Das trennt uns vom Telefon, benötigt aber weiterhin die `.so`-Datei und native Emulator-Bibliotheken.
3. `arm64mini` **(reiner Python-Interpreter):** Ein eigens geschriebener AArch64-Interpreter führt den echten Maschinencode der Bibliothek Instruktion für Instruktion in reinem Python aus. Das ist das Rückgrat von `lipy`.
4. **Lift (handgeschriebene Python-Reimplementierung):** Die Tabellen-Operationen der White-Box werden direkt als kompakte Python-Funktionen ausformuliert. Während der vollständige statische Lift ein Forschungswerkzeug blieb, sind seine **elf Leaf-Primitive** (die kleinsten Bausteine der White-Box-Runde) in `lipy` produktiv aktiv: Als schneller Python-Hook ersetzen sie im Interpreter jeden nativen Blatt-Aufruf durch direkten Python-Code (§9 und §13).

Der Begriff „`.so`-frei“ hat damit eine präzise Bedeutung: Es rechnet weiterhin die echte Logik der White-Box, allerdings interpretiert in Python statt nativ auf einer ARM-CPU. Dadurch entfallen das Smartphone, externe C-Emulatoren und durch den späteren Speicher-Snapshot (§11) schließlich auch die `.so`-Datei selbst.

Verlässliche Ground Truth: Jede dieser vier Ebenen wurde Byte für Byte gegen die jeweils darunterliegende Ebene geprüft: Die Python-Leaves gegen den Interpreter, der Interpreter gegen den C-Emulator und dieser gegen echte Mitschnitte vom Telefon. Wie dieser Python-Interpreter im Inneren konstruiert ist, zeigt das folgende Kapitel.

6. Der reine Python-ARM64-Interpreter

> Eine 13 MB große, für Android kompilierte Bibliothek lässt sich nicht ohne Weiteres in Python aufrufen, weil ihr sowohl die ARM-CPU als auch das Betriebssystem fehlen. Dieses Kapitel zeigt, mit welchen minimalen Bausteinen (`arm64mini` und `ale_pure`) wir CPU, Linker und Objekt-Initialisierung in reinem Python nachbilden.

Das technische Herzstück von `lipy` ist ein in reinem Python geschriebener AArch64-Interpreter. Er lädt die Segmente der Bibliothek in einen eigenen virtuellen Adressraum und führt die Krypto-Routinen ohne eine einzige Zeile nativen Code aus.

6.1 Aufbau von Speicher und CPU

- **Seitenbasierter Sparse-Speicher:** Der Adressraum ist in 4-KB-Seiten unterteilt, die erst beim ersten Zugriff (`lazy`) als `bytearray` angelegt werden. Spezialisierte, allokationsfreie Schnellpfade für 8-, 32- und 64-Bit-Zugriffe innerhalb einer Seite legen den Grundstein für eine hohe Ausführungsgeschwindigkeit.
- **CPU-Modell und Instruktions-Cache:** Modelliert sind sämtliche 31 Allzweckregister (`x0` bis `x30` samt 32-Bit-Sicht `w0` bis `w30`), der Stack-Pointer, das Link-Register, die vier `NZCV`-Bedingungsflags sowie die 32 je 128 Bit breiten SIMD/FP-Register (`v0` bis `v31`). Ein Disassembler dekodiert jede Adresse beim ersten Antreffen und legt das Ergebnis in einem Cache ab. Ist dieser Cache einmal gefüllt, läuft der Interpreter später komplett ohne Disassembler (§11).
- **Abgedeckter Befehlssatz:** Implementiert sind rund 130 skalare Operationsformen (Arithmetik, Logik, Shifts, Bitfeld-Operationen, Lade- und Speicherbefehle mit Pre- und Post-Index sowie bedingte Auswahl) plus das benötigte NEON-SIMD-Subset.

6.2 Der Loader und das Replay-Mint („Der Kontext ist die Engine“)

Ein nackter Befehlssatz-Interpreter besitzt noch keinen Betriebssystem-Linker. Der Loader übernimmt deshalb genau die Aufgaben, die sonst Android beim Laden der Bibliothek erledigt:

1. Er kopiert alle `PT_LOAD`-Segmente an ihre virtuellen Zieladressen und wendet die internen Relativ-Relokationen (`R_AARCH64_RELATIVE`) an.
2. Er fängt externe Aufrufe in die C-Standardbibliothek über schlanke Python-Stubs ab (konkret genügen wenige Speicher- und String-Routinen wie `malloc`, `memcpy` und `memset` auf einem einfachen Bump-Allocator).
3. Er schließt teure Laufzeit-Schranken (`RUNTIME_GATE`) und native Lazy-Init-Guards direkt auf den Erfolgs-Statuscode kurz und legt die vorab in Python entpackten Lookup-Tabellen direkt in den emulierten Speicher. Weder ein Thread-lokaler Speicher noch die Go-Laufzeit müssen dafür manuell hochgefahren werden.

Damit die Krypto-Funktionen rechnen können, ist jedoch noch eine zweite Hürde zu nehmen. Die Schlüssel-Objekte der White-Box sind komplexe C++-Objekte mit eigenen Heap-Puffern und Methodentabellen-Zeigern (Vtables). Würde man versuchen, diese Objekte einfach aus dem Speicher eines Smartphones in den Interpreter zu kopieren, würden sämtliche internen Zeiger ins Leere zeigen („Vtable-Objekt-Wall“).

Die Lösung heißt **Replay-Mint**: Statt fremde Speicher-Objekte zu kopieren, führt der Interpreter beim Start die Konstruktoren der Bibliothek (`init_array`) aus und spielt anschließend die originale Initialisierungskette der App nach. Er erzeugt zuerst das zentrale `Engine`-Objekt und prägt danach aus den zwei statischen Blobs der Bibliothek die lebenden Schlüssel-Objekte `data1` und `data2` frisch im eigenen Adressraum. Alle internen Zeiger stimmen damit von selbst. Genau dieser fertig geprägte Zustand wird später im Snapshot (§11) eingefroren.

Zuvor müssen wir jedoch wissen, an welchen Adressen die Krypto-Funktionen liegen und welche Parameter sie beim Aufruf erwarten.

7. Funktions-Identifikation und ABI per Beobachtung

> In einer produktiven Binärdatei fehlen alle Funktionsprototypen und Parameternamen. Übergibt man beim Aufruf auch nur einen Zeiger oder eine Länge im falschen Register, schlägt die Kryptografie fehl. Dieses Kapitel zeigt, wie die exakte Aufrufkonvention (ABI) durch reine Beobachtung am lebenden System hergeleitet wurde.

Sobald der Interpreter den Code laden und die Schlüssel-Objekte prägen kann, müssen die eigentlichen Krypto-Funktionen lokalisiert und mit den richtigen Registern und Stack-Werten gefüttert werden.

7.1 Exportierte Funktionen vs. interne Helfer

Die vier exportierten Krypto-Operationen, GCM-Ent- und -Verschlüsselung, Key-Unwrap und RSA-PSS-Signatur, stehen mit Rolle und Offset in der dynamischen Symboltabelle (`.dynsym`) und lassen sich dort direkt ablesen (sie bleibt auch beim Strippen erhalten, sonst wäre die Bibliothek nicht ladbar). Die nicht exportierten internen Helfer des *Replay-Mint* (§6.2) wurden über ihre Rücksprung-Adressen im Live-Trace aufgespürt und im Disassembly bestätigt.

7.2 Die ABI aus reiner Beobachtung

Weil eine native Integritätsprüfung jede Code-Veränderung erkennt, wurde die Aufrufkonvention nicht geraten, sondern am Gerät über Hardware-Breakpoints beobachtet: Am Funktionseintritt liest der Tracer Register, Stack und den Speicher hinter jedem Zeiger gemäß der ARM64-Konvention (AAPCS64) aus, sodass sich die Parameter von selbst zu erkennen geben, Chiffretext, lesbares JSON im Ausgabepuffer, 12 Byte als Nonce/IV, 16 Byte als GCM-Tag und passende Ganzzahlen als Längen. Als Kompass dient ein fester **Erfolgs-Sentinel**: Alle White-Box-Funktionen liefern im Erfolgsfall denselben 32-Bit-Wert und weil hinter `Decrypt` und `Unwrap` die strenge GCM-Tag-Prüfung steht, beweist dieser Rückgabewert sofort, dass jedes Argument korrekt übergeben wurde.

Wie die Funktionen lokalisiert und ihre ABI beobachtet werden, ist in Teil 1 (Live-Hook und Schlüssel-Modell) ausführlich hergeleitet. Für den `.so`-freien Nachbau neu und entscheidend sind allein die zwei folgenden Stolperfallen.

7.3 Zwei lehrreiche Stolperfallen

- **Die Unwrap-Funktion prüft selbst einen GCM-Tag (11 Argumente):** Auf ARM64 werden nur die ersten acht Argumente in den Registern `x0` bis `x7` übergeben. Ab dem neunten Argument wandern sie auf den Stack. Die Unwrap-Funktion nimmt elf Argumente entgegen, darunter auf dem Stack den Zeiger auf den 16 Byte langen GCM-Tag. Als dieser Stack-Platz anfangs mit Nullen belegt war, brach der Aufruf jedes Mal mit einem Fehlercode ab. Erst mit dem echten Tag-Zeiger auf dem Stack lief die Schlüsselableitung durch.

- **Puffer-Kapazität statt Ausgabelänge bei der RSA-Signatur:** Die Signatur-Routine erwartet im sechsten Argument keine zu schreibende Datenlänge, sondern die Kapazität des Ausgabepuffers. Für RSA-2048 muss dort exakt der Wert 256 stehen.

> **Lehre:** Eine durch Speicherbeobachtung gesicherte ABI ist jedem statischen Raten überlegen. Doch als die Funktionen im Python-Interpreter erstmals mit dieser exakten ABI liefen, trat ein rätselhafter Effekt auf: Der entschlüsselte Klartext war zwar lesbar, aber die Funktion meldete trotzdem einen Tag-Fehler. Die Ursache führte direkt zum NEON-Durchbruch in Kapitel 8.

8. Der NEON-Durchbruch

> Ein Interpreter, der nur die gängigen Ganzzahl-Befehle beherrscht, produziert in dieser White-Box scheinbar gültige Ergebnisse, scheitert aber an den Sicherheitsprüfungen. Dieses Kapitel zeigt, warum drei spezifische NEON-SIMD-Mechanismen über Erfolg oder Fehlschlag entscheiden und wie man einen einzelnen fehlerhaften Befehl unter Millionen korrekten Schritten findet.

Zur Erinnerung das Symptom aus dem vorigen Kapitel: Bei der GCM-Entschlüsselung kam zwar lesbarer Klartext heraus, doch der berechnete Authentisierungs-Tag stimmte nicht. Die Funktion meldete also trotz korrektem Klartext einen Fehlschlag. Die Untersuchung dieses Tag-Fehlers förderte einen nicht offensichtlichen Befund zutage: Die eigentliche AES-Schlüsselstrom-Erzeugung der White-Box benötigt keinerlei SIMD-Befehle, weil sie ausschließlich aus skalaren Tabellen-Lookups besteht. An drei sicherheitskritischen Stellen setzt der Code jedoch massiv auf 128-Bit-NEON-Vektoroperationen. Unser anfangs rein skalarer Interpreter lief deshalb „fast richtig“ und wich genau an den entscheidenden Stellen ab:

1. **Interne Objekt-Integritätsprüfung:** Eine interne Prüfroutine verrechnete sich ohne präzise SIMD-Semantik und lehnte frisch geprägte Schlüssel-Objekte sofort als „manipuliert“ ab.
2. **Der 1-Instruktions-Bug im GCM-Auth-Tag:** Bei der GCM-Entschlüsselung stimmte zwar der erzeugte AES-Keystream Byte für Byte, doch der berechnete Authentisierungs-Tag war falsch. Der Fehler steckte in der GF(2^{128})-Multiplikationsschleife der Tag-Finalisierung (GHASH): Sie verwendet einen NEON-Lane-Move (`mov v1.d[1], v0.d[0]`), der eine einzelne 64-Bit-Hälfte zwischen zwei Vektorregistern kopiert. Der Interpreter hatte diesen Befehl fälschlich als Kopie des gesamten 16-Byte-Registers ausgeführt. Ein einziger falsch modellierter Befehl korrumpierte so den gesamten Tag.
3. **Salz-abhängiger Absturz in der RSA-Signatur:** Weil RSA-PSS bei jedem Aufruf einen zufälligen 32-Byte-Salt einmischt, trafen manche Signierläufe datenabhängig auf seltene NEON-Widening-Befehle (`ushll` und Verwandte), die Vektor-Elemente auf die doppelte Bitbreite dehnen. Solange diese Befehlsfamilie im Interpreter fehlte, stürzte die Provisionierung unvorhersehbar bei bestimmten Zufalls-Salts ab. Erst das vollständige Nachrüsten und Unit-Testen aller acht Widening-Varianten machte die Signatur dauerhaft stabil.

Gefunden wurde der 1-Instruktions-Bug im GCM-Tag mit einem eigens gebauten, allgemein nützlichen Diagnosewerkzeug: Einem **resynchronisierenden Trace-Diff** gegen die Ground Truth vom echten Smartphone. Das Werkzeug vergleicht die Ausführungs-Spuren von Interpreter und Gerät Befehl für Befehl, blendet harmlose Unterschiede wie abweichende Speicheradressen von Zeigern automatisch aus und isoliert so aus Millionen identischer Schritte exakt die erste Instruktion, bei der ein Registerwert abweicht. Beim Tag-Bug liefen beide Spuren über **24,3 Millionen Instruktionen** völlig synchron, bevor die GHASH-Schleife den fehlerhaften Lane-Move offenlegte.

> **Lehre:** In der Kryptografie ist „läuft fast“ gleichbedeutend mit „funktioniert nicht“. Erst mit der bit-genauen NEON-Emulation rechnete der Interpreter identisch zur echten Hardware und erlaubte es, das innere Tabellenwerk der White-Box vollständig aufzuklären. Die praktische Konsequenz daraus ist eine opcode-genaue Unit-Test-Suite gegen die Referenzsemantik: Ein gezielter Test, der etwa prüft, dass ein Lane-Move genau eine 64-Bit-Hälfte und nicht das ganze Register kopiert, hätte genau diesen Fehler sofort aufgedeckt. Solche Tests sichern die Engine seither ab.

9. Die White-Box ausführbar reproduziert

> Wer die White-Box nur blind als Ganzes emuliert, verschenkt Geschwindigkeit und versteht nicht, welcher Algorithmus im Inneren überhaupt arbeitet. Dieses Kapitel öffnet die Box, macht anhand der Tabellen-Aufrufe und mehrerer unabhängiger Indizien plausibel, dass es sich um 14-rundiges AES-256 handelt und ersetzt die innersten Schleifen durch schnelle Python-Funktionen.

Nachdem Teil 1 die White-Box von außen beschrieben hatte, wurde ihre innere Struktur in diesem Schritt bis auf den einzelnen Tabellen-Zugriff zerlegt und in Python reproduziert.

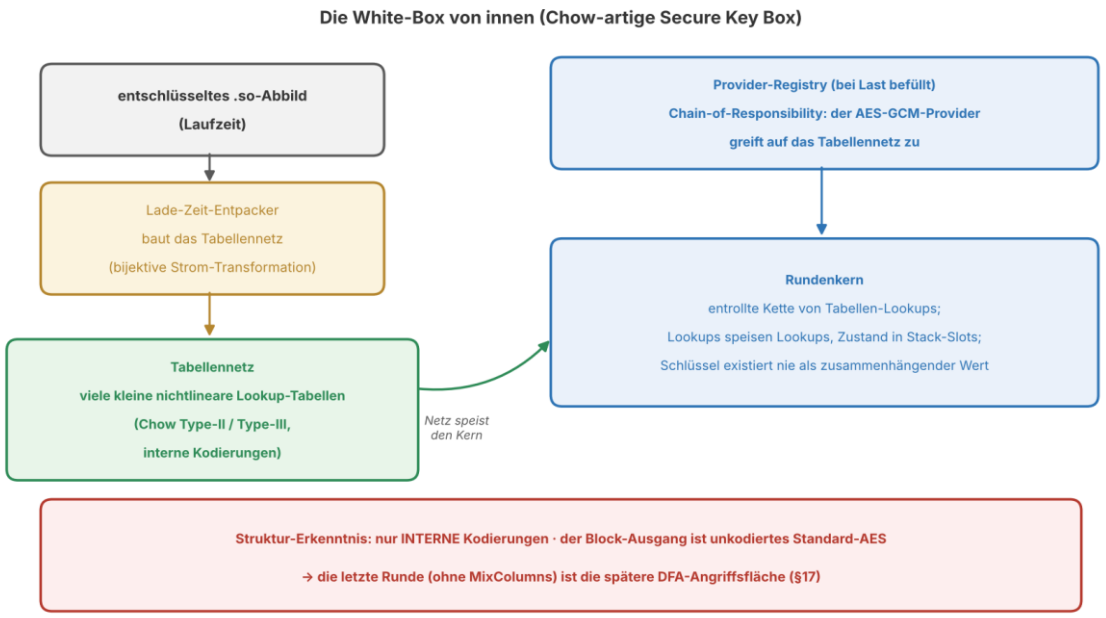


Abbildung 5: Die White-Box von innen: Ein Lade-Zeit-Entpacker baut aus dem entschlüsselten Abbild ein Netz kleiner nichtlinearer Lookup-Tabellen (Chow Type-II/III). Eine Provider-Registry erreicht über eine Chain-of-Responsibility den AES-GCM-Provider, der den entrollten Rundenkern speist. Der Block-Ausgang ist unkodiertes Standard-AES.

Die Analyse zeigt eine gehärtete **Chow-artige White-Box-Konstruktion** [2], die aus drei Bausteinen besteht:

- **Tabellen und Lade-Zeit-Entpacker:** Der Schlüssel existiert nie im Klartext, sondern ist in 3-Bit-Slices zerlegt und in drei großen Lookup-Tabellen im Speicher eingebacken. Die erste Tabelle (ein 10-Bit-Netz aus 240 × 1.024 Einträgen) wird beim Laden aus einem XOR-maskierten Bitstrom entpackt. Ein eigener Python-Nachbau (`_unpackA`) erledigt diese Entpackung in wenigen Zeilen direkt beim Laden. Unmittelbar dahinter liegen zwei weitere Tabellen (8-Bit und 12-Bit), auf denen die Blockfunktion arbeitet.
- **Der Rundenkern:** Er besteht aus einer eingabe-unabhängigen Kette von rund 188 Tabellen-Lookups und ließ sich über tausende Zufallsvektoren mit null Abweichungen in Python abspielen.

- **Die AES-256-Blockfunktion und ihre elf Leaf-Primitive:** Ein kompletter 16-Byte-Blockschritt unter dem statischen Schlüssel `data2` ruft insgesamt **14.336 Blatt-Funktionen (Leaves)** auf. Da eine einzelne Runde exakt 1.024 solcher Aufrufe benötigt (14.336 = 14 × 1.024), liefert das Zählen dieser Aufrufe ein starkes Indiz für **14 Runden und damit AES-256** [6]. Gestützt wird das durch die erkannte AES-Rundenstruktur, das unkodierte Standard-AES am Ausgang der letzten Runde (§17) und das Standard-GCM-Aufrufmuster. Die Rundenzahl allein wäre kein Beweis, da auch andere Chiffren 14 Runden besitzen können. Obwohl ein Block über zehntausend Aufrufe tätigt, nutzt der Code dafür immer wieder dieselben **elf Leaf-Formen**. Alle elf Formen wurden byte-exakt in Python nachgebaut und als Fast-Hook in den Interpreter eingehängt, über eine einfache Tabelle, die jede der elf Blatt-Adressen auf die zugehörige Python-Funktion abbildet. Sobald der Maschinencode eine dieser Adressen anspringt, übernimmt sofort die schnelle Python-Funktion.

Ein entscheidendes Detail: In-Place-Aliasing ist Teil der Semantik. Beim Nachbau der elf Leaves trat anfangs in rund 9 bis 12 Prozent der Aufrufe eine Byte-Abweichung auf, obwohl die mathematische Formel exakt dem Disassembly entsprach. Die Ursache war *In-Place-Aliasing*: In diesen Aufrufen übergibt das Programm für den Ausgabepuffer dieselbe Adresse wie für einen der Eingabepuffer. Besitzt ein Leaf einen späteren Misch-Schritt, der den Eingabepuffer erneut liest, sieht er dort auf der echten CPU bereits die kurz zuvor selbst geschriebenen Ausgabebytes statt der ursprünglichen Eingabe. Erst als dieses Verhalten in den Python-Leaves exakt nachgebildet war, liefen sämtliche Testvektoren fehlerfrei durch.

Mit den gehookten Leaves war die White-Box im Python-Interpreter schnell und vollständig lauffähig. Das folgende Kapitel zeigt nun den kryptografischen Hebel, der dafür sorgt, dass wir diese White-Box im laufenden Betrieb fast überhaupt nicht mehr aufrufen müssen.

10. Der entscheidende Hebel: SK2 ist Standard-AES-256-GCM

> Müsste der Python-Interpreter für jede minütliche Glukose-Messung die White-Box durchlaufen, wäre der Client im Alltag träge und ressourcenhungrig. Dieses Kapitel erklärt die zentrale Reduktion des Projekts: Warum die White-Box pro Sitzung nur für zwei einzige 16-Byte-Blöcke gebraucht wird und jede Glukose-Entschlüsselung danach nur noch 0,02 ms dauert.

Der wichtigste Befund für den Produktivbetrieb lautet: Nur der Transport des Sitzungsschlüssels ist durch die White-Box (`data2`) geschützt. Der daraus entpackte 32-Byte-Schlüssel **SK2** ist dagegen ein ganz gewöhnlicher AES-256-Schlüssel und alle Glukose-Antworten des Servers sind **Standard-AES-256-GCM** [6, 7] unter `SK2`. Sobald `SK2` einmal vorliegt, wird für das Lesen der Glukosewerte überhaupt keine White-Box mehr benötigt.

Warum der Kanal auf eine Standard-Chiffre auflöst. Dass ausgerechnet AES-256-GCM zum Einsatz kommt, ist kein Zufall, sondern strukturell angelegt: Die native Bibliothek verdrahtet ihre Kryptografie nicht fest, sondern löst sie über eine interne **Algorithmen-Registry** auf. Dies ist ein Katalog von Fabriken für standardisierte kryptografische Primitive, vergleichbar mit einem JCE-Provider oder einer OpenSSL-ENGINE-Tabelle. Jede Krypto-Operation wählt darüber ihre Implementierung anhand einer Algorithmenkennung. Eine rein lesende Analyse dieser Registry zeigte, dass sowohl der Transport des Sitzungsschlüssels als auch der Glukose-Kanal auf genau benannte Standard-Verfahren verweisen, nicht auf eine proprietäre Eigenchiffre. Die White-Box kapselt also nur den *Schlüssel-Transport*. Die eigentliche Nutzdaten-Verschlüsselung ist dem Namen nach gewöhnliches AES-256-GCM. Das ist der tiefere Grund, warum Phase B ohne jede White-Box auskommt, die Bibliothek selbst behandelt den Glukose-Datenstrom als Lehrbuch-GCM.

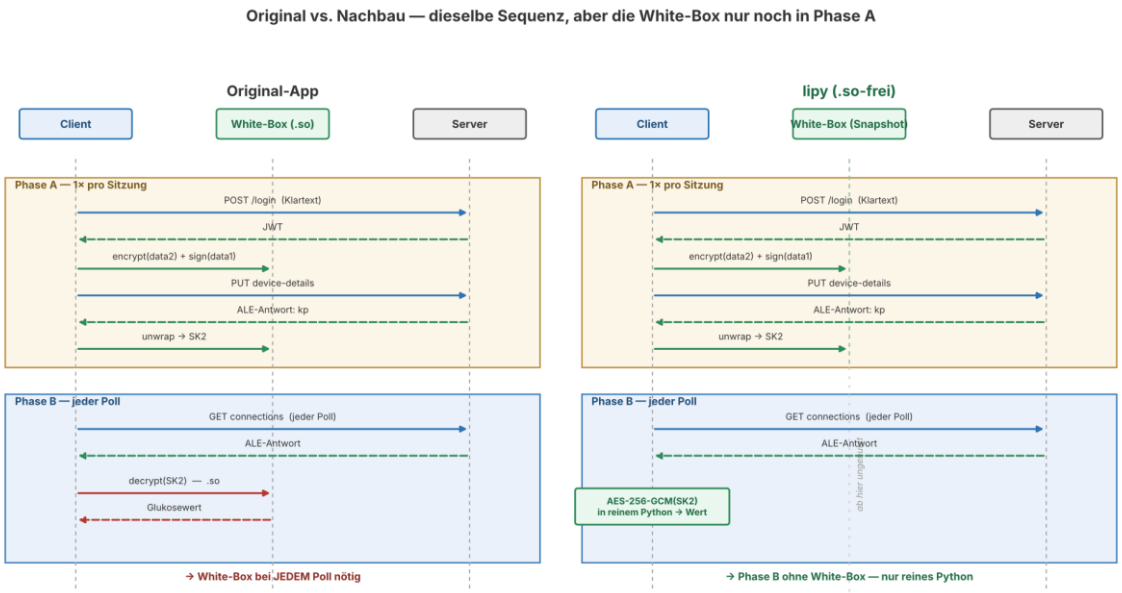


Abbildung 6: Original vs. Nachbau, dieselben Server-Aufrufe, aber die White-Box nur noch einmalig in Phase A statt bei jedem Lesen.

Weil AES-GCM intern im Zählermodus (CTR) arbeitet und seinen Schlüsselstrom durch Verschlüsseln aufsteigender Zählerblöcke (`Nonce ‖ Counter`) erzeugt, lässt sich auch das Auspacken von `SK2` drastisch abkürzen. Statt die komplexe `gcm_unwrap`-Funktion mit ihren elf Argumenten aufzurufen, genügt die isolierte 16-Byte-Blockfunktion der White-Box (`eblock`). Daraus ergibt sich die **Zwei-Phasen-Architektur** von `lipy`:

- **Phase A (einmal pro Sitzung):** Die Server-Antwort auf den Registrierungs-Handshake (~2,3 KB) ist mit `data2` verschlüsselt und enthält an ihrem Ende das eingewickelte Schlüsselpaket. Statt die gesamte Antwort in rund 31 s durch die White-Box zu schicken, entschlüsselt der Schnellpfad (*Tail-CTR*) über die Blockfunktion nur die letzten Blöcke am Ende der Nachricht, liest daraus das eingebettete 32-Byte-Paket aus und gewinnt `SK2` anschließend mit **exakt zwei White-Box-Blockaufrufen** in insgesamt rund **1,5 s**. Nur für diesen kurzen Moment wird die White-Box überhaupt benötigt.
- **Phase B (beliebig oft im Dauerbetrieb):** Jede Glukose-Antwort wird mit einer Standard-AES-256-GCM-Routine (aus der Python-Bibliothek `cryptography`) entschlüsselt. Das dauert rund **0,02 ms** pro Abruf und kommt völlig ohne `.so`-Datei, ohne White-Box und ohne Gerät aus.

Kryptografisch selbstbeweisende Korrektheit: Seit der AEAD-Härtung prüft auch der Schnellpfad in Phase A den GCM-Authentifizierungs-Tag des eingebetteten Schlüsselpakets über die GHASH-Prüfsumme und einen zusätzlichen Blockaufruf nach und bricht bei Abweichung ab. Diese Prüfung bezieht sich konkret auf das entschlüsselte Schlüsselpaket mit seiner Nonce, dem zugehörigen Tag und dem geladenen Ciphertext-Bereich. Der Tail-CTR-Pfad entschlüsselt zwar nur den für die Schlüsselgewinnung benötigten Ausschnitt, prüft den Tag aber über das vollständige Paket. Eine pauschale Aussage über das gesamte Protokoll ist damit nicht verbunden. Phase B bestätigt den gewonnenen Schlüssel anschließend praktisch: Entschlüsselt `SK2` das erste reale Glukose-Paket mit gültigem 128-Bit-GCM-Tag, dann ist er für diese Nachricht mit überwältigender Wahrscheinlichkeit der korrekte Schlüssel. Ein auch nur um ein einziges Bit falscher Schlüssel würde den Tag mit einer Fälschungswahrscheinlichkeit von rund 2^{-128} nicht bestehen.

Damit ist die Arbeit der White-Box auf das einmalige Bereitstellen von `data1` und `data2` in Phase A geschrumpft. Kapitel 11 zeigt nun, wie wir selbst für diese Phase A auf die `.so`-Datei verzichten können.

11. Der Snapshot statt der .so

> Das Laden der ELF-Datei, das Entpacken der Tabellen und das Prägen der Schlüssel-Objekte (§6) dauern im Interpreter mehrere Minuten und würden weiterhin die originale `.so`-Datei voraussetzen. Dieses Kapitel zeigt, wie ein eingefrorener Speicher-Snapshot den Start auf einen Sekundenbruchteil verkürzt und die `.so`-Datei endgültig überflüssig macht.

Weil die in der White-Box eingebetteten Objekte `data1` und `data2` über alle Sitzungen hinweg statisch sind, muss ihr teurer Aufbau nicht bei jedem Programmstart wiederholt werden. Das Artefakt, das die `.so`-Datei zur Laufzeit ersetzt, ist deshalb ein **Snapshot**: Ein serialisierter, LZMA-komprimierter Abzug des Interpreter-Speichers unmittelbar nach dem einmaligen Aufbau des statischen Materials.

Statt die Bibliothek bei einer neuen Registrierung erneut zu laden, liest der Client diesen Snapshot in einem Sekundenbruchteil ein. Da im Snapshot neben den Speicherseiten und den Objekt-Zeigern auch die bereits fertig dekodierten Instruktionen aus dem Disassembler-Cache gespeichert sind, läuft der Client im Betrieb nicht nur `.so`-frei, sondern auch komplett ohne Disassembler-Bibliothek (`capstone`).

Der Begriff „Instruktions-Cache“ meint dabei die dekodierten Befehle, die der Disassembler beim ersten Antreffen jeder Adresse ablegt; „warm“ heißt, dieser Cache ist aus einem früheren Lauf bereits gefüllt. Entscheidend ist, was eingefroren werden darf und was nicht: Die Speicherseiten unmittelbar nach der Initialisierung und der Dekodier-Cache hängen allein am unveränderlichen Programmcode (`.text`) und sind damit statisch. Der Laufzeit- und Engine-Zustand dagegen, den eine RSA-Signatur unterwegs verändert, darf nicht in den Snapshot gelangen (Abbildung 7).

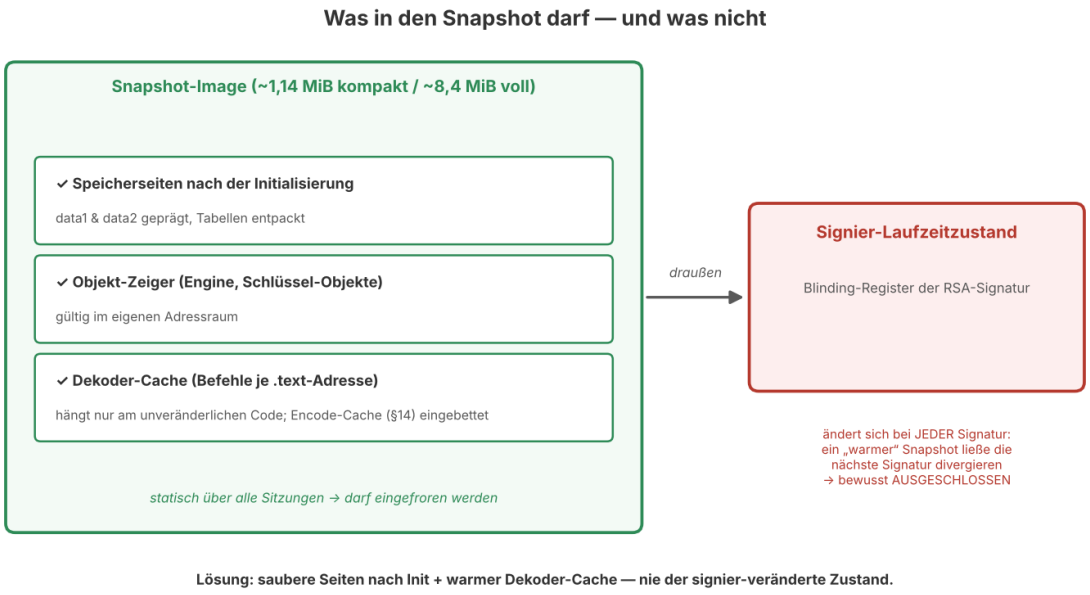


Abbildung 7: Was statisch in den Snapshot darf und was nicht.

> **Fallstrick aus der Praxis: Warum der erste „warme“ Snapshot defekt war.** Um den Instruktions-Cache auch für die komplexe RSA-Signatur vorzuwärmen, wurde der Snapshot anfangs einfach nach einem kompletten Test-Signierlauf gespeichert. Dieser Snapshot erwies sich jedoch als unbrauchbar: Der Signierpfad verändert während seiner Arbeit internen Zustand im Arbeitsspeicher der White-Box. Startete man aus diesem „verschmutzten“ Zustand eine zweite Signatur, wichen die Ergebnisse ab, was zunächst fälschlich wie ein Bug im JIT-Compiler aussah.

Erst der isolierende Gegentest (ein frischer Snapshot nach der Initialisierung funktioniert, ein Snapshot nach einer Signatur schlägt fehl) brachte die Ursache ans Licht. Die saubere Lösung kombiniert beide Welten: Man nimmt die **unberührten Speicherseiten direkt nach der Initialisierung** und fügt ihnen ausschließlich den **warmen Instruktions-Cache** aus einem Signierlauf hinzu. So startet der Snapshot sauber und ist trotzdem ab der ersten Sekunde voll vorgewärmt.

In mehreren Schritten schrumpfte dieser Fußabdruck. Ein vollständiges Signieren liest nur rund 8 % der Snapshot-Seiten, der Rest wird nie berührt. Beschränkt man das Image auf diesen tatsächlichen Lese-Arbeitssatz, fällt es vom vollen Arbeits-Snapshot (rund 8,4 MiB) zunächst auf ein 1,36-MiB-Minimal-Image und schließlich auf das standardmäßig verwendete, selbsttragende Call-0-Image von rund 1,135 MiB, das den Encode-Cache (§14) und den nötigen Laufzeit-Kontext bereits eingebettet trägt. Dieses Image lädt auf dem Arbeits-Laptop in rund 0,1 s. Die anschließende Schlüssel-Entfaltung (Unwrap) braucht rund 1,76 s, ein Bruchteil des ursprünglichen Mehr-Minuten-Aufbaus.

Mit diesem Snapshot kann der Client nun völlig eigenständig gegen die echte Cloud antreten. Welche Hürden das Live-Protokoll dabei bereithält, beschreibt Kapitel 12.

12. Die .so-freie Provisionierung und der Client

> Eine Entschlüsselung, die im Labor mit aufgezeichneten Testdaten funktioniert, wird vom echten Server oft trotzdem abgewiesen. Dieses Kapitel dokumentiert die drei entscheidenden Protokoll-Hürden bei der Live-Registrierung, die erst im Zusammenspiel mit dem echten Backend sichtbar wurden.

Der fertige Python-Client (`GlucoClient`) führt die gesamte Kette allein aus E-Mail-Adresse und Passwort durch: Er meldet sich an, provisioniert bei Bedarf einmalig die Sitzung aus dem Snapshot und liest anschließend im Dauerbetrieb die Glukosewerte. Beim Übergang vom Offline-Test zum Live-Server waren drei Punkte entscheidend, die als allgemeine Lehren für Protokoll-Analysen gelten können:

1. **Die RSA-Signatur muss den vollständigen Inhalt abdecken, nicht nur einen Präfix:** In einem frühen Offline-Mitschnitt sah es so aus, als würde nur ein 20 Byte langer Zeitstempel-Präfix signiert. Eine so gebildete Anfrage wies der Server jedoch jedes Mal ab. Erst die Signatur über die gesamten 77 Byte (Präfix plus kompletter JSON-Body) wurde akzeptiert. Zwei unabhängige, live verifizierte Implementierungen bestätigten genau diese Form. Das zeigt, wie leicht ein einzelner Offline-Capture in die Irre führen kann, solange man ihn nicht gegen den Live-Server prüft.
2. **Der signierte Zeitstempel braucht eine Punktlandung in beide Richtungen:** Weil die RSA-Signatur im Python-Interpreter anfangs über eine Minute dauerte, wäre ein mit der aktuellen Uhrzeit gebildeter Zeitstempel beim Absenden der Nachricht bereits abgelaufen. Der Client setzt den Zeitstempel deshalb um eine Sicherheitsmarge in die Zukunft, berechnet die Signatur und wartet per `time.sleep` exakt bis zu diesem Zielzeitpunkt ab. Dabei lauerte ein subtiler Fehler in beide Richtungen: Ist die Marge zu klein, kommt das Paket zu spät an und wird abgelehnt. Umgekehrt hatte eine frühe Version des Codes eine Obergrenze beim Warten (`wait_s <= 90`). Als die Engine durch spätere Optimierungen (§13) deutlich schneller wurde, war die Restwartezeit plötzlich größer als 90 s, die Warteschleife wurde übersprungen und der Client schickte einen Zeitstempel *in der Zukunft*. Auch das lehnt der Server sofort ab. Erst das bedingungslose Warten auf den Zielzeitpunkt kombiniert mit einer dynamisch aus der letzten Laufzeit gemessenen Marge löste das Problem dauerhaft. Zudem wird die nachrichtenunabhängige GCM-Verschlüsselung des Handshake-Bodys vor die Zeitstempel-Berechnung gezogen, sodass die Marge am Ende nur noch die reine `pss_sign`-Dauer (nach §14 rund 6,4 s) abdecken muss.

3. **Durchgängige Konsistenz der Installations-Kennung (`iid`):** Der Server liefert das Schlüsselpaket (`kp`) beim Handshake nur dann aus, wenn eine aktive Registrierung stattfindet. Bei einer bereits registrierten Kennung bleibt die Antwort leer. Gleichzeitig muss die selbst erzeugte Installations-Kennung über Login, Handshake und alle späteren Datenabrufe hinweg strikt identisch zur Kennung (`iid`) im ausgestellten Login-JWT gehalten werden, da der Server die Sitzung sonst mit einem Fehlercode abweist.

Dass der Server diesen allein aus dem generischen Snapshot gebauten Handshake akzeptiert, beweist zugleich praktisch, dass die Schlüssel `data1` und `data2` in jeder App-Installation identisch sind (§20). Bleibt noch eine praktische Frage für den Alltag: Wie verkürzt man die Wartezeit bei dieser einmaligen Registrierung auf ein angenehmes Maß?

13. Performance-Engineering

> Ein reiner Python-Interpreter ist von Natur aus langsam. Die erste lauffähige RSA-Signatur dauerte rund 25 Minuten, die erste komplette Registrierung über fünf Minuten. Dieses Kapitel zeigt, durch welche gezielten Eingriffe die einmalige Provisionierung auf dem Referenz-Desktop auf rund 120 s gedrückt wurde, ohne ein einziges Ausgabebyte zu verändern. Mit der Signatur-Beschleunigung aus §14 sind daraus auf dem Arbeits-Laptop heute wenige Sekunden Rechenzeit geworden (siehe Kapitelende).

Um den Interpreter für die Praxis schnell genug zu machen, griffen zwei Optimierungsebenen ineinander: Eine allgemeine Beschleunigung der Engine und eine gezielte Optimierung des Provisionierungs-Handshakes aus §12.

- **Allgemeine Engine-Ebene:** Ein **Basic-Block-JIT** übersetzt geradlinige Folgen von ARM64-Befehlen bis zum nächsten Sprung einmalig in Python-Funktionen und führt sie danach als Block aus. Allein dieser Hebel beschleunigte die RSA-Signatur um etwa den Faktor 6 (auf dem Test-Laptop von rund 1.499 s auf 248 s). Hinzu kommt der Python-Fast-Hook der elf White-Box-Leaves aus §9, der die Tabellen-Schleifen etwa um den Faktor 15 beschleunigt.
- **Gezielte Optimierung der Provisionierung (unter Standard-CPython):** Auf dem Referenz-Desktop benötigte die einmalige Provisionierung in der ersten lauffähigen Version noch rund **320 s tatsächliche Gesamtdauer** (190 s Krypto-Berechnung plus 128 s Sicherheitspuffer bis zum in die Zukunft gesetzten Zeitstempel). Drei streng regressionsgeprüfte Maßnahmen brachten diese Zeit auf **rund 120 s** herunter.

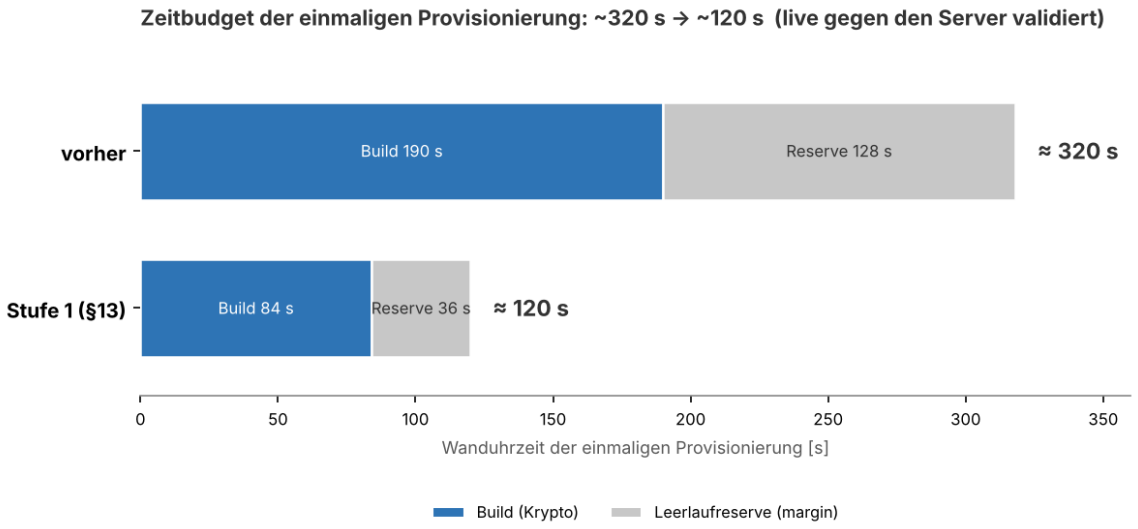


Abbildung 8: Zeitbudget der einmaligen Provisionierung vorher/nachher (Referenz-Desktop): Der Build sinkt von rund 190 auf 84 s, die Leerlaufreserve von 128 auf 36 s und die tatsächliche Gesamtdauer damit von rund 320 auf etwa 120 s, live gegen den Server bestätigt.

Die drei Hebel hinter Abbildung 8 im Einzelnen:

1. **Request-GCM über die isolierte Blockfunktion:** Statt die Payload-Verschlüsselung des Handshakes durch den vollen Interpreter laufen zu lassen, nutzt auch sie direkt die schnelle Blockfunktion aus §10 kombiniert mit GHASH in reinem Python. Die Dauer sinkt dadurch von **25 bis 30 s auf 1,44 s**.

2. **Montgomery-Leaf-Hook und nachgewärmter Snapshot für** `pss_sign`**:** Ein Profiling der RSA-Signatur zeigte zwei Zeitfresser: Das nachgelagerte, maskierte Rechenwerk der Signatur (~24 % der Laufzeit, das spätere Werk 2 aus §19). Außerdem die innerste CIOS-Montgomery-Multiplikation [8, 9] des offenen Werks (~18 % der Laufzeit, rund 4.850 Aufrufe pro Signatur). Die Montgomery-Multiplikation rechnet modular ohne teure Division. „CIOS“ ist ihre schleifenverschränkte Standardvariante (siehe Glossar). Während das maskierte Werk hier zunächst unangetastet blieb und erst in §14 über register-genaue Stubs beschleunigt wird, wurde das mathematisch klar definierte Montgomery-Primitiv in Python gehookt (`out = a · b · R⁻¹ mod n`). Zusätzlich wurden die internen RSA-Fenstertabellen im Snapshot vorab entpackt (Prewarm). Dadurch fiel die kalte Erst-Signatur von rund 111 s auf 78,8 s und der gesamte Krypto-Build von 190 s auf 84 s.
3. **Nachführen der Zeitstempel-Marge:** Da der Build nun zuverlässig in 84 s durchläuft, konnte das in die Zukunft gesetzte Zeitfenster (§12) von 320 s auf 120 s reduziert werden. Die reine Wartezeit bis zum Absenden sank damit von **128 s auf 36 s**.

Das Ergebnis wurde live gegen den echten Server bestätigt (`HTTP 200` beim PUT-Request, gefolgt von einer korrekten `SK2`-Ableitung und einem gültigen Glukosewert). Der minütliche Poll im Alltag ist von alldem ohnehin unberührt, da er in 0,02 ms über Standard-AES-256-GCM läuft (§10).

Einordnung der Messwerte: Die großen Sprünge sind durchweg Software-Effekte, nicht Hardware. Der Faktor 6 (rund 1.499 s → 248 s) stammt allein vom Block-JIT und wurde als erstes lauffähiges Interpreter-Light auf einem Laptop ganz ohne weitere Optimierung gemessen. Alle feiner abgestimmten Zahlen (Krypto-Build 190 s → 84 s, kalte Signatur 111 s → 78,8 s, Gesamtdauer 320 s → 120 s) wurden einheitlich auf dem Referenz-Desktop erhoben. Der schnellere Rechner liefert dabei nur einen konstanten Grundfaktor, während die eigentlichen Reduktionen aus den beschriebenen Hebeln kommen (JIT, Leaf-Hook, isolierte Blockfunktion, Montgomery-Hook, Prewarm). Der Kaltstart-Wert von 111 s enthielt bereits den Montgomery-Hook (er hatte die warme Signatur zuvor von 129 s auf 78 s gesenkt). Der Schritt auf 78,8 s zeigt isoliert den Gewinn des nachgewärmten Snapshots.

Auf dem Laptop läuft die einmalige Provisionierung heute, mit der Signatur-Beschleunigung aus §14 und dem vorgewärmten Snapshot aus §11, in rund 7 s Krypto-Build und rund 2,3 s Schlüsselableitung (SK2). Die Zeitstempel-Marge kalibriert sich dabei selbst auf rund 18 s. Aus den anfänglichen Minuten sind damit wenige Sekunden Rechenzeit geworden, die einmalig bei der Registrierung anfallen.

Damit ist der Client schnell und einsatzbereit. Kapitel 14 treibt dieselbe Optimierung in Stufe 2 weiter und beschleunigt das maskierte zweite Signierwerk samt Nachrichten-Vorlauf bis auf rund 6,4 s reine Signaturzeit; Kapitel 15 klärt anschließend, was zu tun ist, wenn der Hersteller eine neue App-Version veröffentlicht.

14. Register-genaue NEON-Stubs: Der beobachtbare Signierpfad wird praxistauglich

> §13 war Stufe 1 derselben Produktions-Optimierung: Es ersetzte das offene Montgomery-Primitiv des ersten Signierwerks (§19) durch Python und wärmte den Snapshot vor, was die einmalige Provisionierung auf rund 120 s brachte. Mit weiterhin aktivem Montgomery-Hook verblieb die warme Einzelsignatur auf dem Arbeits-Laptop dennoch bei rund 138 s, denn den Löwenanteil trägt nun das maskierte zweite Werk. Dieses Kapitel ist Stufe 2: Ein korrektes Zeit-Profil legt den wahren Engpass in wenigen NEON-Schleifen offen und register-genaue Python-Stubs plus ein Encode-Cache senken die Signatur von rund 138 s über rund 17 s und rund 13 s bis auf rund 6,4 s, ohne ein einziges Ausgabebyte zu verändern.

Der Montgomery-Hook aus §13 beschleunigt das offene erste Werk der Signatur, die Fenster-Exponentiation (§19) und bleibt in allem Folgenden aktiv. Genau deshalb kostet dieses erste Werk im späteren Zeit-Profil nur noch Sekundenbruchteile und der verbleibende Löwenanteil liegt im maskierten zweiten Werk, das §13 bewusst unangetastet gelassen hatte. Mit aktivem Hook stand die warme Signatur auf dem Arbeits-Laptop bei rund 138 s, zu langsam für den Alltag und mühsam für die Fault-Kampagnen, die Tausende Signaturen rechnen. Die 78,8 s aus §13 und diese 138 s sind dabei nicht direkt vergleichbar: Dort ging es um die kalte Erst-Signatur auf dem schnelleren Referenz-Desktop, hier um die warme Signatur auf dem langsameren Arbeits-Laptop, auf dem in diesem Kapitel alle Stufen einheitlich gemessen sind. Die Hebel dieses Kapitels sind dabei selbst wieder Python-Hooks, nur eine Ebene tiefer, mitten in der Funktion.

Der blinde Fleck: Dispatch-Zahl statt Zeit. Das erste Profiling zählte, wie oft jeder Basic Block betreten wird und wies die Spitzenreiter als „erledigt“ aus, ohne Wirkung auf die Gesamtlaufzeit. Grund ist eine Eigenheit der Engine: Ein Block, der eine kurze, auf sich selbst zurückspringende Schleife enthält, wird vom Block-JIT zu einem Durchlauf zusammengefasst. Der Dispatch-Zähler registriert dann einen Eintritt, obwohl der Block intern mehrere hundert Iterationen abarbeitet. Drei heiße Blöcke mit zusammen rund 67.600 gezählten Eintritten entsprachen so in Wahrheit bis zu 34,6 Millionen inneren Iterationen. Die Rangliste nach Eintritten war nicht ungenau, sondern irreführend, denn sie zeigte genau an den teuersten Stellen vorbei.

> Die verlässliche Währung ist nicht die Eintrittszahl, sondern die tatsächlich verstrichene Zeit pro Programmstelle. Erst ein Wall-Clock-Profil pro Basic Block statt der Dispatch-Zählung machte den Engpass sichtbar.

Der Befund. Das Zeit-Profil war eindeutig: Rund 79 % der Signierzeit steckten in drei nahezu identischen NEON-Schleifen, ein weiterer Anteil in einer vektorisierten Byte-Schleife (Abbildung 9). Jede der drei Schleifen wird etwa 8.200-mal betreten und rechnet dabei je rund 37 innere Iterationen dichter 128-Bit-Vektorarithmetik, die Bignum-Masken-Arithmetik des zweiten Signierwerks (§19). Für genau diese SIMD-Befehle besitzt der Interpreter keinen schnellen Pfad und fällt pro Einzelbefehl in die allgemeine Emulation zurück. Multipliziert mit zig Millionen Iterationen ergibt das die beobachteten Minuten. Wie schon beim NEON-Durchbruch in §8 gilt: Die Kosten liegen nicht in exotischen Befehlen, sondern in gewöhnlichen Vektoroperationen, die extrem oft laufen.

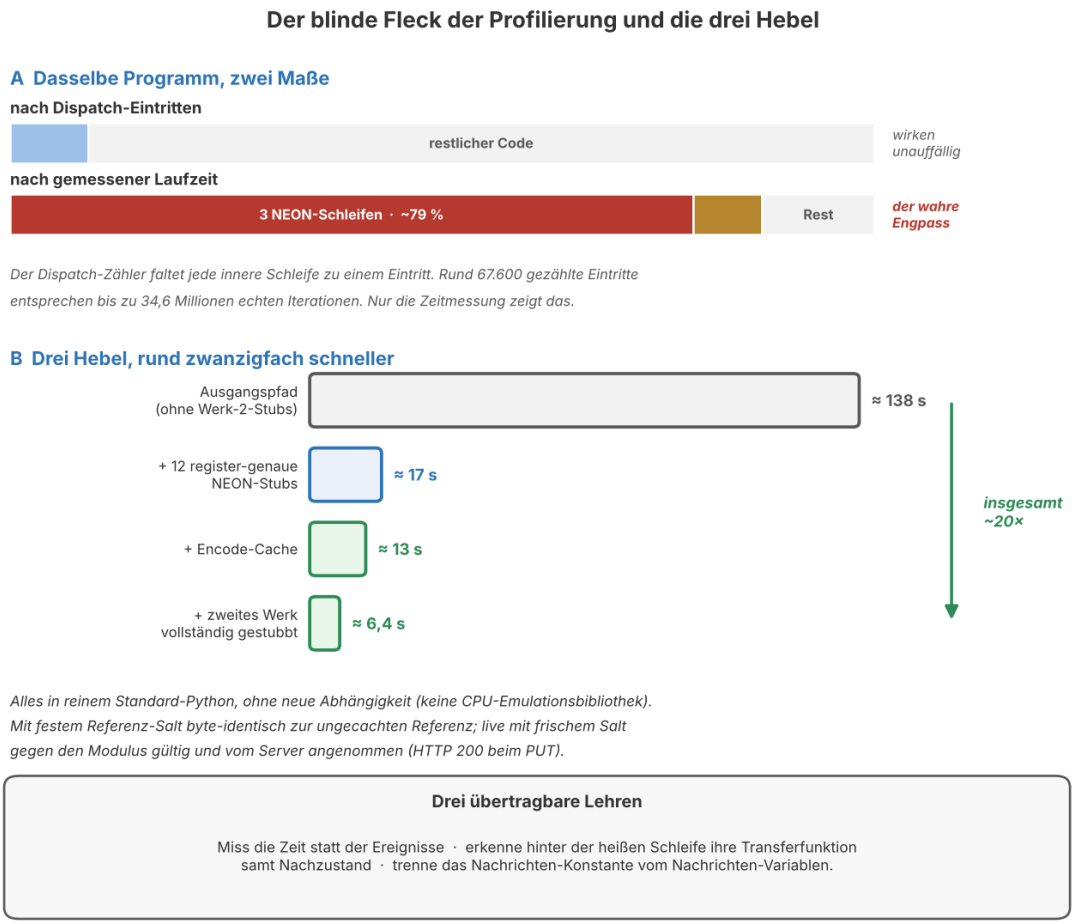


Abbildung 9: Der blinde Fleck der Profilierung. Nach Dispatch-Eintritten wirken die drei heißen NEON-Schleifen unauffällig, nach gemessener Laufzeit tragen sie rund 79 %. Rechts der Beschleunigungs-Verlauf von rund 138 s über die register-genauen Stubs (~17 s) bis zum Encode-Cache (~13 s) und dem vollständig gestubbten zweiten Werk (~6,4 s).

Der Hebel: Register-genaue Mid-Function-Stubs. Eine Schleife, die millionenfach dieselbe Vorschrift auf wandernde Operanden anwendet, ist eine wohldefinierte Transferfunktion ihrer Register. Statt sie Mikroschritt für Mikroschritt zu emulieren, wurde für jede der wenigen heißen Schleifen eine kompakte Python-Entsprechung aus dem Disassemblat des Speicherabbilds abgeleitet, nicht geraten, sondern aus dem tatsächlich laufenden Maschinencode. Das ist dieselbe Idee wie der Montgomery-Leaf-Hook aus §13, nur eine Ebene tiefer, nämlich mitten in der Funktion. Drei Feinheiten entscheiden über Erfolg:

- **Register- und Flag-genauer Nachzustand.** Es genügt nicht, das Schleifenergebnis zu liefern. Der Folgecode erwartet Schleifenzähler, Zeiger und Statusbits exakt so, wie die echte Schleife sie hinterlassen hätte. Ein `cbnz`-Austritt setzt keine Flags, ein `cmp; b.ne`-Austritt hingegen schon. Wird das verwechselt, entgleist die nachfolgende Logik lautlos.
- **Sauberer Wiedereinsprung.** Der Stub gibt die Kontrolle an genau die Instruktion zurück, an der die Schleife natürlich endet: Liefert er eine Zieladresse, wird dorthin gesprungen, andernfalls fällt die Ausführung regulär auf die Rücksprungadresse.
- **Zwillinge unterscheiden.** Die drei NEON-Blöcke sehen einander zum Verwechseln ähnlich, tragen aber unterschiedliche Datenzeiger. Nach jedem Stub wurde neu profiliert. Identische innere Iterationszahlen entlarven Klone und zeigen den jeweils nächsten Engpass.

Zwölf solcher Mid-Function-Stubs für die heißen Schleifen brachten die Einzelsignatur von rund 138 s auf rund 17 s, ein Faktor von etwa 8, vollständig in reinem Python und ohne eine neue Abhängigkeit, insbesondere ohne CPU-Emulationsbibliothek.

Der zweite Hebel: Encode-Cache. Ein erneutes Zeit-Profil nach den Stubs zeigte, dass ein großer Teil der verbleibenden Arbeit nicht von der zu signierenden Nachricht abhängt. Nur die PSS-Kodierung des Nachrichtenblocks ändert sich von Aufruf zu Aufruf. Die schwere Vorbereitung der internen Tabellen ist jedes Mal dieselbe. Daraus wurde ein Delta-Verfahren: Der nachrichtenunabhängige Zustand wird einmal festgehalten und je Signatur wird nur noch der kleine nachrichtenabhängige Unterschied neu gerechnet und der Rest aus dem gespeicherten Zustand repliziert. Entscheidend ist, dass dieses Replay nebenwirkungsfrei und idempotent ist, denn der Cache wird nicht verbraucht, sodass beliebig oft signiert werden kann. Der Encode-Cache drückte die Signatur von rund 17 s auf rund 13 s, womit der pure Pfad an dieser Stelle rund zehnmal schneller ist als die Ausgangsmessung.

Der dritte Hebel: Das zweite Werk vollständig stubben. Ein letztes Zeit-Profil nach dem Encode-Cache zeigte, dass die verbliebene Zeit fast ausschließlich im maskierten zweiten Werk steckt, nicht mehr nur in den drei bereits ersetzten NEON-Schleifen, sondern verteilt über alle neun heißen Blöcke seiner Hauptschleife samt der vier MAC-Kerne. Für sie wurde dieselbe Technik eine Stufe weitergetrieben: Register-genaue Python-Entsprechungen für jeden der neun Blöcke, abgeleitet aus dem laufenden Maschinencode. Die vier MAC-Kerne schienen zunächst zu billig für einen eigenen Eingriff. Das Zeit-Profil widerlegte jedoch den Augenschein: Nicht ihre geringe Codegröße, sondern ihre zehntausendfach durchlaufenen ganzzahligen Faltungsschleifen und Einzelbefehls-Emulationen trugen spürbar zur Laufzeit bei. Erst als auch diese Kerne zu je einem gefalteten Stub zusammengefasst wurden, fiel die warme Signatur auf dem Arbeits-Laptop von rund 13 s auf rund 6,4 s (auf dem schnelleren Referenz-Desktop aus §13 unter 4 s), bit-für-bit identisch zur Referenz, weiterhin in reinem Python und ohne jede CPU-Emulationsbibliothek. Damit ist der pure Signierpfad insgesamt rund zwanzigmal schneller als die erste lauffähige Messung.

Validierung. RSA-PSS mischt pro Signatur einen frischen 32-Byte-Zufalls-Salt ein, sodass jede Signatur anders aussieht. Mit festem Referenz-Salt ist die über den Encode-Cache erzeugte Signatur Bit für Bit identisch zur ungecachten Referenz. Im Live-Betrieb mit frischem Salt verifiziert sie gegen den öffentlichen Modulus und wird vom Server angenommen (HTTP 200 beim PUT, korrekte SK2-Ableitung, gültiger Glukosewert), konsistent mit der Produktionsvalidierung aus §16. Der minütliche Alltags-Poll bleibt davon ohnehin unberührt (§10).

> Lehre: Miss die Zeit, nicht die Ereignisse. Erkenne hinter einer heißen Schleife ihre Transferfunktion samt dem Zustand, den sie hinterlässt. Und trenne das Nachrichten-Konstante vom Nachrichten-Variablen, bevor du optimierst. Die drei Hebel gelten für jede Emulations- und Beschleunigungsaufgabe und machten hier den voll beobachtbaren Signierpfad erst alltagstauglich. Damit ist der Signierpfad so schnell wie der übrige Client. Das folgende Kapitel wendet sich der Wartung über App-Updates hinweg zu.

15. Offset-Migration und Wartung

> Ein Reverse-Engineering-Projekt verliert schnell seinen Wert, wenn jedes App-Update wochenlange Neuanalysen erzwingt. Dieses Kapitel zeigt anhand eines realen Versionssprungs, warum die meisten Updates überhaupt keine Anpassung an der Engine erfordern und wie man bei echten Änderungen systematisch vorgeht.

Damit ein unabhängiger Client dauerhaft nutzbar bleibt, muss der Wartungsaufwand nach einem App-Update gering und planbar sein. Dabei hat sich in der Praxis (beim realen Update von Version 5.1.1 auf 5.2.0) gezeigt, dass man zwei völlig unterschiedliche Update-Fälle trennen muss:

Fall 1: Reines Hüllen-Update (der beobachtete Normalfall)

Beim Versionssprung der App (5.1.1 auf 5.2.0) rotierte der DexGuard-Packer zwar sämtliche Dateinamen der Bibliotheken sowie die Rohschlüssel, das 16-Bit-Maskenmuster der Schlüsselableitung, die Opaque-Predicate-Skalare und die ShiftRows-Verdrahtung der äußeren Verpackung. Sobald unser automatischer Offline-Extraktor (§4.2) die gepackten Dateien direkt aus den jeweiligen APK-Archiven entpackt hatte, besaß das entschlüsselte White-Box-ELF in beiden Versionen jedoch exakt denselben SHA-256-Hash. Ein neues Schloss um denselben Tresor.

> **Aufwand:** Nahezu null. Ein einziger Lauf des Offline-Extraktors bestätigt über den identischen Hash, dass der Krypto-Kern unverändert geblieben ist. Der bestehende Snapshot läuft ohne jede Offset-Migration weiter. Lediglich die Versionsnummer im HTTP-Header wird angepasst.

Fall 2: Echtes Neukompilat der nativen Bibliothek

Erst wenn der Hersteller den C++/Go-Code der White-Box tatsächlich neu übersetzt, verschieben sich die internen Funktions- und Tabellen-Adressen, während die Logik gleich bleibt („Semantik stabil, Adressen instabil“). Für diesen Fall existiert ein vierstufiges, weitgehend automatisiertes Playbook:

1. **Neu entpacken:** Der Offline-Extraktor (§4) kalibriert die rotierenden Packer-Parameter vollautomatisch über die ELF-Kennung.
2. **Exportierte Offsets auslesen:** Ein Skript liest die dynamische Symboltabelle (`.dynsym`) des neuen ELF aus und liefert sofort die neuen Offsets der vier exportierten Krypto-Funktionen.
3. **Interne Helfer und Tabellen verankern:** Die wenigen nicht exportierten Helfer und Tabellen-Adressen werden über Live-Trace-Rücksprünge bzw. Muster-Scans lokalisiert und mit dem Leaf-Validator byte-genau gegen das neue Binary geprüft.
4. **Opcode-Audit und neuer Snapshot:** Ein Audit-Skript prüft, ob das neue Kompilat zusätzliche ARM64-Befehle nutzt, baut den Snapshot neu und prüft die gesamte Kette mit dem Offline-Validator gegen Referenz-Captures.

> **Konsequenz:** Weil im Alltag meist nur die äußere Packer-Hülle rotiert und selbst für ein Neukompilat feste Werkzeuge bereitstehen, kostet ein App-Update Minuten bis wenige Stunden statt Wochen neuer Forschung. Wie all diese Bausteine in ein sauberes Software-Paket gegossen wurden, zeigt Kapitel 16.

16. Produktionsreife und Validierung

> Lose Forschungsskripte eignen sich weder für den stabilen Dauerbetrieb noch für eine saubere Trennung zwischen eigenem Code und fremdem geistigen Eigentum (IP). Dieses Kapitel beschreibt die Zwei-Zonen-Architektur von `lipy` und die zweistufige Validierung.

Aus den Analyse-Werkzeugen entstand mit `lipy` ein strukturiertes Python-Paket (sowie parallel die native Android- und Wear-OS-App `GlucoBridge`, die den Wert direkt auf die Smartwatch bringt). Das Repository folgt einer strikten **Zwei-Zonen-Trennung**:

- **Veröffentlichbare, IP-freie Laufzeit:** Sie umfasst Konfiguration, sicheren lokalen Speicher, den Hintergrund-Poller mit Backoff, die SQLite-Verlaufsdatenbank, die API-Anbindung sowie CLI und

Desktop-GUI. Dieser gesamte Teil enthält keinerlei herstellerspezifische Konstanten, Offsets oder Binärdaten.

- **Lokale Engine- und Secret-Zone:** Sie kapselt den Interpreter, die White-Box-Anbindung und den Snapshot. Diese Zone wird über `.gitignore` und einen automatischen **Pre-Commit-Guard** (`ip_guard.py`) strikt aus dem Git-Repository ferngehalten. Zudem wird sie im Code *lazy* und *absent-safe* geladen: Fehlt die Engine-Zone (etwa auf einem schlanken Smart-Home-Server, auf den nur der einmalig erzeugte `SK2`-Cache kopiert wurde), läuft der Client im Dauerbetrieb trotzdem uneingeschränkt weiter.

Validiert ist das Paket auf zwei Ebenen: **(A) Offline** über 46 automatisierte Unit-Tests. Weiterhin über einen Byte-Validator, der Verschlüsselung, Signatur, Schlüsselableitung und Glukose-Entschlüsselung ohne Netzwerk, Byte für Byte gegen echte Captures prüft. Sowie **(B) live** im End-to-End-Betrieb gegen den Server.

> **Überleitung zur wissenschaftlichen Kür (§17 bis §19):** Mit dem produktiven Client ist das eigentliche Projektziel vollständig erreicht. Doch weil wir mit dem eigenen Python-Interpreter volle Kontrolle über jeden einzelnen Rechenschritt der White-Box haben, können wir nun zwei Fragen beantworten, die über das reine Auslesen hinausgehen: Wie widerstandsfähig ist die eingesetzte White-Box-Mathematik eigentlich gegen gezielte Fehlerangriffe (Fault Injection), mit denen Angreifer normalerweise versuchen, die eingebackenen Schlüssel im Klartext zu extrahieren. Lässt sich der Schlüssel am Ende vielleicht sogar direkt aus dem Signierwerk selbst herauslösen?

17. Fault-Analyse A: DFA gegen die AES-White-Box

> Ein Software-Interpreter erlaubt es, gezielt einzelne Bits mitten in einer Berechnung zu kippen, um aus den entstehenden Rechenfehlern auf den geheimen Schlüssel zurückzuschließen. Dieses Kapitel unterzieht die AES-256-White-Box diesem Härtetest: Es zeigt, wie der letzte Rundenschlüssel K_{14} vollständig fällt, warum der vorletzte Rundenschlüssel K_{13} jedoch standhält und was das über die Qualität der White-Box verrät.

Im `.so`-freien Interpreter lassen sich interne Zwischenwerte der Blockfunktion nicht nur lesen, sondern an jedem beliebigen der 14.336 Leaf-Schritte gezielt stören. Das Ziel dieser **differentiellen Fault-Analyse (DFA)** ist es, die in den Tabellen versteckten Rundenschlüssel von `data2` zu bergen. Für einen vollständigen Bruch von AES-256 benötigt man die letzten beiden 16-Byte-Rundenschlüssel K_{14} und K_{13}. Dies ist ein reiner Stresstest der Krypto-Mathematik (für den Betrieb von `lipy` wird er nicht benötigt) und bereitet die Sicherheits-Lehre in §20 mit vor.

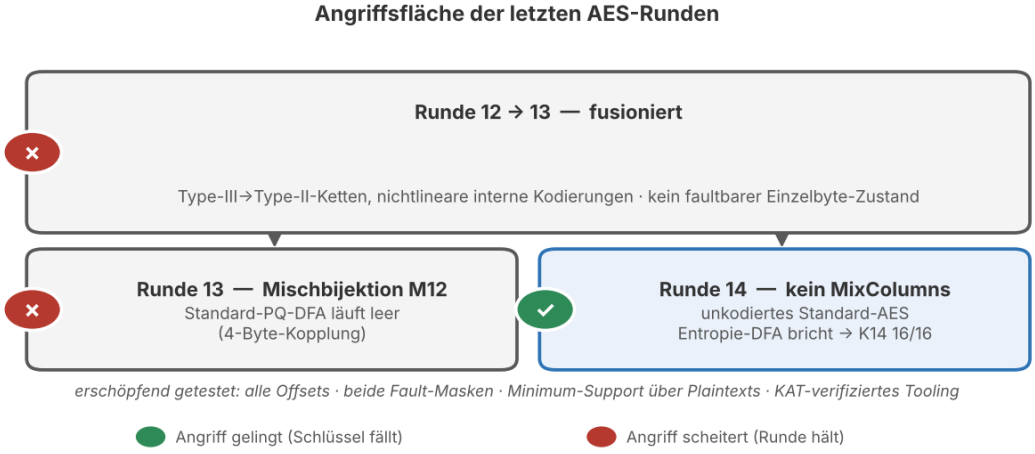


Abbildung 10: Angriffsfläche der letzten AES-Runden: Nur die letzte Runde (ohne MixColumns) ist für Single-Fault-DFA offen. Die vorletzte Runde und die fusionierte Grenze davor halten.

Die systematische Untersuchung zeigte drei gestaffelte Verteidigungslinien der White-Box:

1. **Seitenkanalanalyse am Eingang (DCA [11]) ohne Signal:** Eine Differential Computation Analysis am Beginn der Runde lieferte kein verwertbares Signal, weil eine 16-Byte-Eingangsvormischung sofort ab dem ersten Blatt alle Eingabebytes miteinander koppelt.
2. **Klassische DFA auf die vorletzte Runde (Runde 13) läuft ins Leere:** Der bekannte Piret-Quisquater-Diagonalangriff [4] setzt voraus, dass ein einzelner Byte-Fehler vor dem vorletzten `MixColumns` als charakteristischer 4-Byte-Differenzvektor (2δ, δ, δ, 3δ) am Ausgang erscheint. Die White-Box koppelt die vier Spaltenbytes jedoch über eine 32-Bit-Mischbijektion (M_{12}), sodass ein einzelner Slice-Flip keinen sauberen Einzelbyte-Fehler erzeugt.
3. **Der Durchbruch an der letzten Runde (Runde 14 liefert K_{14} vollständig mit 16/16 Bytes):** Die letzte AES-Runde besitzt laut Standard kein `MixColumns` und ihr Ausgang muss unkodiertes Standard-AES sein. Stört man einen 3-Bit-Slice unmittelbar vor der letzten SubBytes-Schicht, bleibt die entstehende Differenz am S-Box-Eingang über viele verschiedene Klartexte hinweg in einem winzigen, niedrigdimensionalen Unterraum beschränkt.

Ein **Entropie-Solver** prüft daraufhin für jede Byte-Position j alle 256 möglichen Schlüsselkandidaten k ∈ {0, …, 255} und zählt über alle Fault-Paare (C, C*), wie viele verschiedene Rückwärts-Differenzen vor der S-Box auftreten:

$$\text{distinct}_k = |\{\, \text{INV_SBOX}[C[j] \oplus k] \oplus \text{INV_SBOX}[C^*[j] \oplus k] \;\mid\; \text{alle Fault-Paare } (C, C^*) \,\}|$$

Für den wahren Rundenschlüssel-Kandidaten k = K_{14}[j] bricht diese Zahl auf die kleine Unterraumgröße ein (nur 3 bis 7 verschiedene Werte), während sie für alle 255 falschen Kandidaten hoch bleibt (11 bis 26 Werte). Auf diesem Weg fielen **sämtliche 16 Bytes des letzten Rundenschlüssels K_{14}** mit deutlicher Trennschärfe (3× bis 6× Abstand zum zweitbesten Wert) und stabil über verschiedene Zufalls-Seeds.

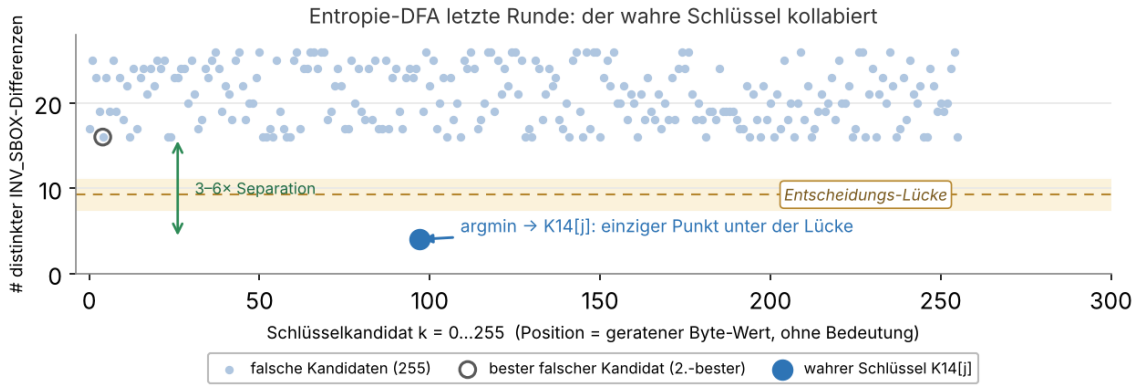


Abbildung 11: Entropie-Kollaps je Byte-Position: Abgelesen wird der wahre Schlüssel als eindeutiges Minimum unterhalb der Lücke, nicht über seine Position auf der x-Achse.

Wie man den Treffer abliest: Entscheidend ist nicht die Position auf der x-Achse, sie ist nur der geratene Byte-Wert und trägt keine Bedeutung, sondern allein die Höhe: Der korrekte Schlüssel ist der eine Kandidat, dessen distinct-Zahl als klares Minimum unter die Lücke fällt, während alle übrigen im oberen Band bleiben.

Warum die vorletzte Runde (K_{13}) standhält: Mit bekanntem K_{14} lässt sich die letzte Runde mathematisch invertieren, um nach einem sauberen Einzelbyte-Fault in der vorletzten Runde zu suchen. Obwohl das gesamte innere Band (2.700 Leaf-Positionen über alle sechs Offsets, sowohl mit Einzelbit- als auch Voll-Byte-Fehlern) erschöpfend abgetastet und zusätzlich mit einer konventions-freien Gegenprobe direkt auf das Piret-Quisquater-Muster {δ, δ, 2δ, 3δ} getestet wurde, trat an keiner einzigen Stelle ein solches Muster auf (0/16 Bytes).

Auch die statistischen Seitenkanäle blieben im getesteten Rahmen ohne Signal. Über den Eingang hinaus wurde die Differential Computation Analysis (DCA) [11] systematisch gegen den Rundenschlüssel geführt und zwar erster, zweiter und dritter Ordnung, über mehrere tausend Durchläufe, mit vollständiger Rangliste aller 256 Kandidaten und z-Wert. Unter diesen dokumentierten Mess- und Auswertebedingungen trat in keiner Ordnung ein statistisch signifikantes Signal für den untersuchten Rundenschlüssel auf. Der wahre Schlüssel lag statistisch symmetrisch um null, ununterscheidbar von den falschen Kandidaten. Das belegt keine generelle DCA-Immunität der Konstruktion, wohl aber, dass die naheliegende DCA unter den getesteten Bedingungen kein verwertbares Leck lieferte.

> **Deutung:** Die Grenze zur vorletzten Runde ist in der White-Box vollständig zu nichtlinear-kodierten Tabellenketten (Type-III zu Type-II) verschmolzen. Der dortige Zwischenzustand existiert zu keinem Zeitpunkt als einzeln störbares Byte im Speicher. Eine Single-Fault-DFA kann K_{13} daher prinzipiell nicht bergen. Auch der klassische algebraische Weg (BGE [3]) greift nicht unmittelbar: Er setzt byteweise 8→8-Bit-Bijektionen voraus, doch die White-Box verbreitert den Zustand (10 statt 8 Bit) und führt ihn in 3-Bit-Slices mit Übertragskette, die Standard-BGE-Annahme trifft nicht zu. Übrig bliebe allein eine vollständige, mehrwöchige Rückgewinnung der Kodierungen eigener Größenordnung (BGE-Klasse).

Damit ist die Bilanz des AES-Zweigs klar: Der letzte Rundenschlüssel fällt, der vorletzte hält und sowohl die untersuchte DCA (bis dritter Ordnung) als auch der klassische algebraische Weg (BGE) blieben im getesteten Rahmen ohne verwertbares Leck. Mit derselben Methode prüft Kapitel 18 nun den asymmetrischen RSA-Signaturzweig.

18. Fault-Analyse B: RSA-PSS und warum non-CRT den Bellcore-Angriff aushebelt

> Bei fast allen RSA-Implementierungen genügt ein einziger gestörter Signiervorgang, um den privaten Schlüssel über den berühmten Bellcore-Angriff komplett zu berechnen. Dieses Kapitel zeigt das überraschende Ergebnis unseres zweiten Fault-Experiments: Warum die RSA-White-Box der App langsam ist, aber genau deshalb gegen diesen Angriff immun bleibt.

Analog zur AES-Untersuchung stellte sich die Frage, ob sich der in `data1` versteckte private RSA-2048-Schlüssel d durch Fehlerinjektion während der Registrierungs-Signatur (`pss_sign`, RSA-PSS nach PKCS#1 [10]) extrahieren lässt.

Der klassische **Bellcore-Angriff** [5] (Boneh, DeMillo und Lipton, 1997) greift nicht die abstrakte RSA-Mathematik an, sondern die in der Praxis fast immer genutzte **CRT-Beschleunigung** (Chinesischer Restsatz): Statt $s = m^d \bmod N$ direkt über den großen Modulus $N = p \cdot q$ zu berechnen, rechnet eine CRT-Implementierung getrennt modulo den beiden Primfaktoren p und q ($s_p = m^d \bmod p$ und $s_q = m^d \bmod q$) und setzt beide Teilergebnisse am Ende zusammen. Trifft ein injizierter Fehler nur eine dieser beiden Hälften, so liefert der größte gemeinsame Teiler aus der Differenz von korrekter Signatur s und fehlerhafter Signatur ŝ sofort einen der geheimen Primfaktoren:

$$\gcd(s - \hat{s},\ N) = p$$

Mit p ist der Modulus N faktorisiert und der gesamte private Schlüssel aus einer einzigen fehlerhaften Signatur geborgen.

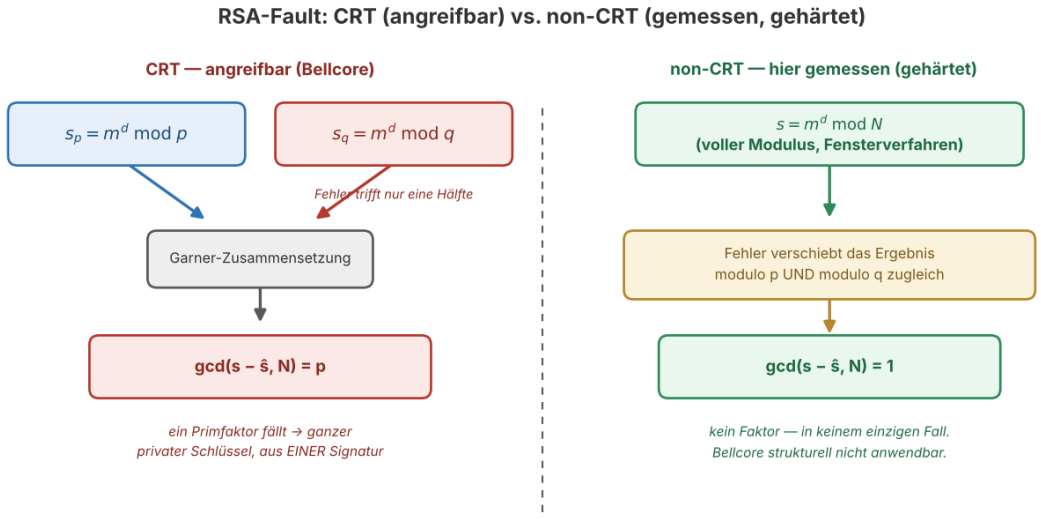


Abbildung 12: CRT (links) ist angreifbar: Ein Fehler in einer Modul-Hälfte isoliert den Primfaktor p. Der hier gemessene non-CRT-Pfad (rechts) stört die volle Restklasse. Der größte gemeinsame Teiler liefert nur 1 und keinen Faktor.

Im Python-Interpreter wurden an zahlreichen Stellen der Signaturberechnung (zwischen 26 % und 98 % des Rechenfortschritts) gezielt Fehler in die Montgomery-Multiplikation eingestreut. Das Ergebnis lautete in jedem einzelnen Fall ausnahmslos:

$$\gcd(s - \hat{s},\ N) = 1$$

Es fiel kein einziger Faktor heraus. Die Erklärung liefert die Struktur des untersuchten Codes (§13): Die White-Box verzichtet vollständig auf die CRT-Optimierung und berechnet die Signatur **non-CRT** als eine einzige modulare Exponentiation über den vollen 2048-Bit-Modulus N. Ein Rechenfehler verschiebt das Ergebnis dadurch modulo p und modulo q gleichzeitig. Die Differenz s - ŝ teilt keinen gemeinsamen Faktor mit N, womit die Grundvoraussetzung des klassischen Bellcore-GCD-Angriffs bei den untersuchten Fehlerstellen gar nicht erst existiert. Das zeigt die Robustheit gegen genau diese Angriffsklasse. Eine allgemeine Immunität gegen beliebige Fehlermodelle ist damit nicht behauptet.

> **Deutung:** Die non-CRT-Umsetzung opfert massiv Rechengeschwindigkeit (sie ist der Grund, warum `pss_sign` in §13 so aufwendig optimiert werden musste), wirkt dafür aber als eingebaute, strukturelle Gegenmaßnahme gegen den klassischen Bellcore-GCD-Angriff. Zusammen mit dem AES-Befund aus §17 zeigt sich damit, dass beide kryptografischen Zweige gegen die hier untersuchten klassischen Fault-Extraktionen widerstandsfähig sind. Eine allgemeine Fault-Immunität ist damit nicht gezeigt.

Kapitel 19 öffnet nun das Signierwerk selbst und zeigt, warum der geheime Exponent neun direkten Analysewegen standhielt, bevor Kapitel 20 die übergeordnete Sicherheits-Lehre zieht.

19. Das private Signierwerk: Warum die untersuchten Live-Extraktionen den geheimen Exponenten nicht preisgeben

> §18 hat gezeigt, dass die Signatur non-CRT als `m^d mod N` über den vollen Modulus rechnet und der Bellcore-Angriff damit ausfällt. Die naheliegende Anschlussfrage lautet: Liegt der private Exponent `d` irgendwo in der Berechnung offen? Dieses Kapitel beschreibt die White-Box-Konstruktion des Signierwerks, einen offen gerechneten und einen maskiert gerechneten Teil und zeigt an neun konkreten Extraktions-Ansätzen, warum `d` in den untersuchten Spuren zu keinem Zeitpunkt als lesbare Größe auftrat. Der offen bestimmbare Anteil des Exponenten ließ sich dabei vollständig rekonstruieren, der geheime nicht. Der Schlüssel wurde nicht extrahiert. Für das Projektziel, die Interoperabilität mit den eigenen Daten, ist das weder nötig noch beabsichtigt.

Zwei Werke, ein Zwischenwert. Extern ist die Signatur eine einzige modulare Exponentiation `m^d mod N` (§18). Intern zerfällt sie in zwei deutlich getrennte Werke, verbunden über genau einen übergebenen Zwischenwert (Abbildung 13). Der Schlüssel steckt in einer additiven Spaltung des Exponenten: `d` ist in einen offen verwendeten Anteil und einen geheimen Korrekturanteil zerlegt.

- **Werk 1, offen.** Eine klassische Montgomery-Fenster-Exponentiation über den vollen Modulus rechnet den offenen Anteil (rund 4.850 Montgomery-Multiplikationen, non-CRT, passend zum Befund aus §18) und übergibt einen einzigen Zwischenwert an Werk 2. Dieses Werk trägt kein Geheimnis. Wer hier `d` sucht, sucht am falschen Ort.
- **Werk 2, maskiert.** Es trägt den geheimen Korrekturanteil nach, niemals als nackte Zahl, sondern eingebacken in eine vorberechnete, maskierte Ablauftabelle. Die Rechnung läuft als Montgomery-Multiplikation RNS-artig über rund 150 parallele Spuren und knapp 55 Schritte, allerdings nicht über teilerfremde Moduln wie ein klassisches Restklassensystem, sondern über ein redundantes Limb-System. Je Schritt stehen drei Basis-Erweiterungen, genau die drei identischen NEON-Blöcke aus §14. In jeden Schritt mischt ein frisch abgeleiteter Maskenwert, sodass kein Zwischenzustand einer unmaskierten Potenz des Eingangs entspricht.

Was offen liegt, liegt wirklich offen. Der offene Anteil ließ sich tatsächlich vollständig gewinnen: Über ein symbolisches Value-Replay entlang des ersten Werks wurde dessen Exponent rekonstruiert und über zwei verschiedene Nachrichten nachgewiesen, dass er den übergebenen Zwischenwert Bit für Bit reproduziert. Das bestätigt die Rollenverteilung, statt sie zu erschüttern: Das erste Werk trägt ausschließlich den öffentlich bestimmbaren Anteil und kein Bit des Geheimnisses. Der geheime Korrekturanteil bleibt vollständig im maskierten zweiten Werk.

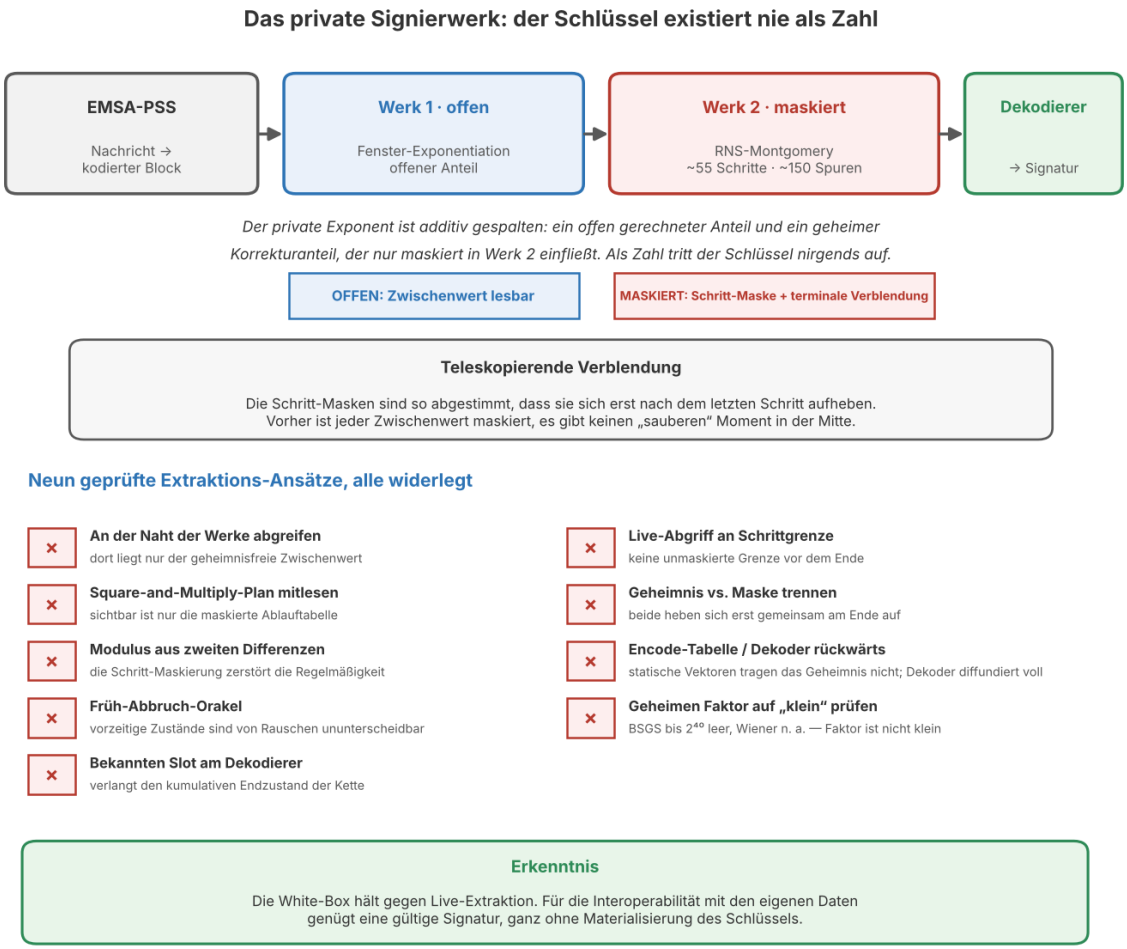


Abbildung 13: Das private Signierwerk. Ein offenes Werk rechnet den offenen Anteil, ein maskiertes RNS-Werk trägt den geheimen Korrekturanteil nach. Eine teleskopierende Verblendung hebt sich erst nach dem letzten Schritt auf. Die neun geprüften Extraktions-Ansätze scheitern jeweils an einer anderen Eigenschaft der Konstruktion.

Die teleskopierende Verblendung. Der Kern der Konstruktion ist, dass die Schritt-Masken nicht unabhängig sind: Sie sind so aufeinander abgestimmt, dass sich ihr Produkt erst nach dem letzten Schritt zu Eins aufhebt. Ein nachgeschalteter Dekodierschritt entfernt die Verblendung schließlich anhand des kumulativen terminalen Zustands der gesamten Kette. Vor diesem Punkt ist jeder beobachtbare Zwischenwert mit einem Maskenanteil behaftet. Es existiert kein „sauberer" Moment in der Mitte, an dem `d` oder eine schlüsselnahe Größe blank vorläge. Die Masken sind dabei nicht als statisches, vorab abgreifbares Band ausgelegt: Sie verweben sich pro Schritt datenabhängig mit dem laufenden Zwischenzustand, sodass sich die Maskenfolge nicht einmal nachrichtenunabhängig vorausberechnen und so abtrennen lässt. Eine register-genaue Gegenprobe am Terminal bestätigt das: Die abschließende Entblendungs-Transformation ist nachrichtenunabhängig und fix. Das einzige nachrichtenabhängige Signal lebt in den Akkumulator-Limbs im Speicher, nicht in einem einzelnen Register. Es gibt also keinen Registerzustand, den man auslesen und invertieren könnte.

Neun Ansätze und warum jeder scheitert. Die Extraktion wurde systematisch gegen die üblichen White-Box-Hebel geprüft. Jeder lief leer. Die Gründe zeigen, was die Konstruktion richtig macht:

- **Die Werke an der Naht auftrennen.** An der Fuge wird nur der geheimnisfreie Zwischenwert übergeben. Der Korrekturanteil betritt die Rechnung erst danach und nie unmaskiert.

- **Den Square-and-Multiply-Plan mitlesen.** Sichtbar ist nicht der Exponent, sondern eine maskierte Ablauftabelle. Die Schrittfolge trägt die Bits von d nicht offen.
- **Den Modulus aus zweiten Differenzen rückrechnen.** Die Maskierung je Schritt zerstört die Regelmäßigkeit, auf der ein solcher Differenzangriff beruht.
- **Ein Früh-Abbruch-Orakel bauen.** Weil die Masken erst terminal zusammenfallen, ist jeder vorzeitig abgegriffene Zustand von Rauschen ununterscheidbar.
- **Am Dekodierer einen bekannten Slot einsetzen.** Der Dekodierschritt verlangt den kumulativen Endzustand der ganzen Kette. Ein einzeln substituierter Slot passt nicht und liefert nichts.
- **An einer sauberen RNS-Schrittgrenze live abgreifen.** Eine unmaskierte Grenze vor dem Ende gibt es nicht. Die Verblendung überspannt lückenlos alle Schritte.
- **Geheimnis und Maske über ihre multiplikative Struktur trennen.** Beide Anteile greifen ununterscheidbar ineinander und heben sich erst gemeinsam am Ende auf.
- **Die Kodier-Tabelle als Transcoder lesen oder den Dekodierer rückwärts abschälen.** Die vorberechneten Kodier-Vektoren sind statisch und nachrichtenunabhängig und tragen den geheimen Anteil nicht offen. Ein schrittweises Rückwärts-Abschälen des Dekodierers scheitert, weil dieser voll diffundiert. Sein Ergebnis hängt vom kumulativen Zustand aller Schritte ab, nicht von einem einzelnen.
- **Den isolierten geheimen Faktor auf einen „kleinen" Wert prüfen.** Weil der offene Anteil vollständig bekannt ist, lässt sich der geheime Faktor rechnerisch freistellen. Doch er ist nachweislich nicht klein, eine Baby-Step-Giant-Step-Suche bis 2^{40} findet ihn nicht und ein Wiener-Angriff [12] (der einen kleinen geheimen Exponenten voraussetzt) greift ebenfalls nicht.

Bilanz und bewusste Grenze. Die Konstruktion hielt gegen alle neun untersuchten Live-Extraktions-Ansätze. Sauber getrennt heißt das: Beobachtet ist, dass diese neun konkreten Angriffe unter den gewählten Bedingungen kein Schlüsselmaterial lieferten. Abgeleitet ist, dass die beschriebenen Konstruktionsmerkmale sie erschweren. Nicht bewiesen ist, dass kein anderes Verfahren den geheimen Anteil gewinnen könnte. Ein theoretischer Weg bleibt, nämlich die gesamte maskierte Kette vollständig und deterministisch nachzurechnen und Schritt für Schritt zu entblenden, doch das ist ein langer, fehleranfälliger Weg und wurde bewusst nicht beschritten. Das Projektziel ist Interoperabilität mit den eigenen Daten. Dafür genügt eine gültige Signatur, die ohne jede Materialisierung von d entsteht. Wie schon in §17/§18 gilt: Befähigung und Nutzen sind zu trennen. Dass der Schlüssel nicht fiel, ist kein Scheitern, sondern das korrekte Ergebnis und zugleich die gewollte Grenze im Umgang mit fremdem geistigem Eigentum.

> Eine tragfähige White-Box verteidigt sich nicht durch ein einzelnes Schloss, sondern dadurch, dass jeder naheliegende Angriff an einer anderen Eigenschaft der Konstruktion scheitert: Getrennte Werke, Exponenten-Spaltung, maskierte Tabelle, Schritt-Masken, terminale Verblendung. Erst ihr Zusammenspiel ließ die untersuchten Live-Extraktionen scheitern. Abbildung 14 ordnet dieses Signierwerk in die Gesamtkarte aller White-Box-Funktionen ein und leitet über zur Sicherheits-Lehre in §20.

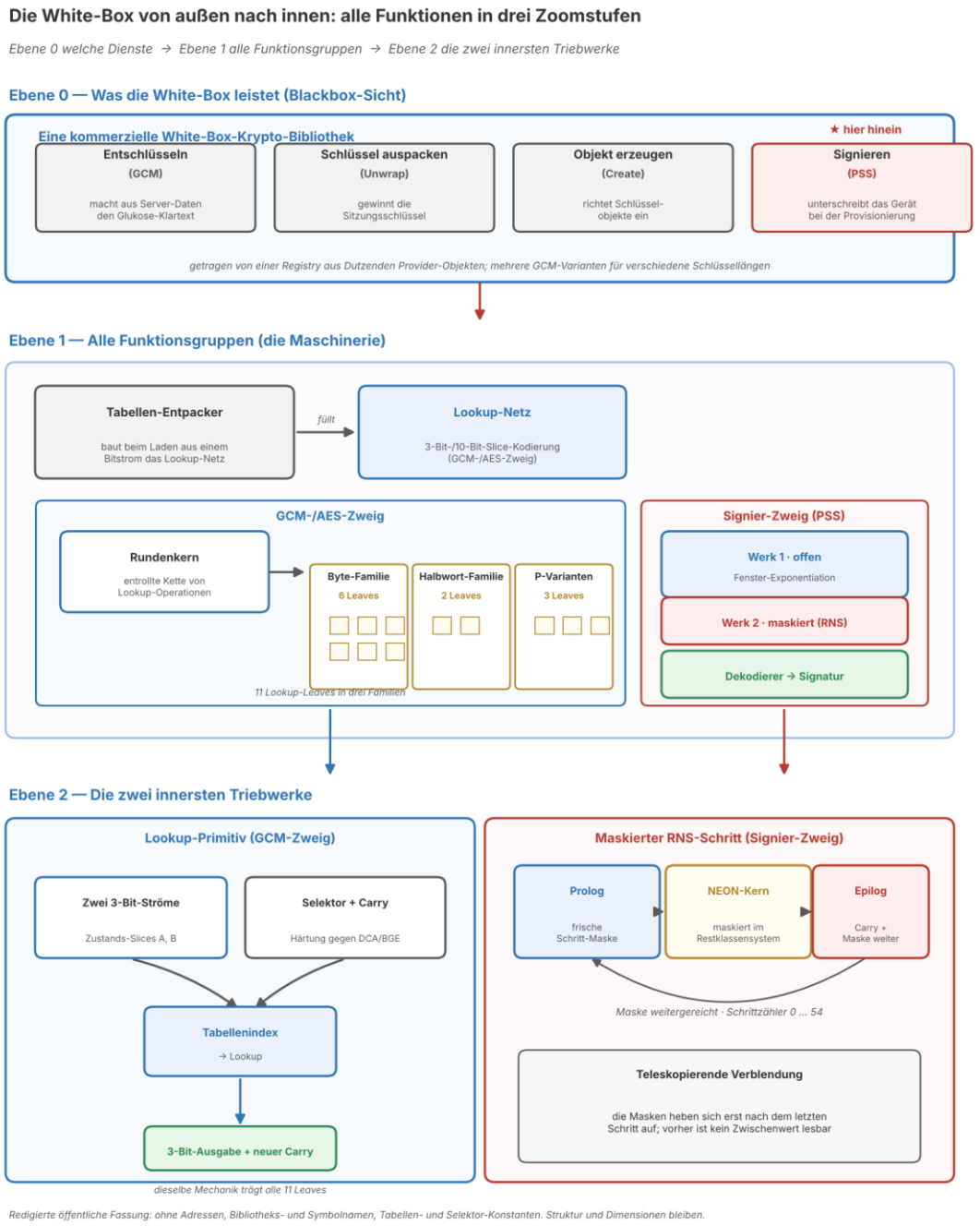


Abbildung 14: Die White-Box von außen nach innen, alle Funktionen in drei Zoomstufen. Ebene 0 die angebotenen Dienste, Ebene 1 sämtliche Funktionsgruppen (Tabellen-Entpacker, Rundenkern, die elf Lookup-Leaves, das Signierwerk), Ebene 2 die zwei innersten Triebwerke. Redigierte öffentliche Fassung ohne Adressen, Bibliotheks- und Symbolnamen sowie Tabellen- und Selektor-Konstanten.

20. Was die Umsetzung zeigt: Die Sicherheits-Lehre

> Hier laufen alle technischen Fäden des Projekts zusammen. Dieses Kapitel beantwortet die übergeordnete Frage für Sicherheitsarchitekten und Entwickler: Warum konnte ein vollständig `.so`-freier Client gebaut werden, obwohl die White-Box-Kryptografie im Inneren allen Angriffen standgehalten hat?

Blickt man auf die Ergebnisse zurück, zeigt sich ein lehrreicher Kontrast: **Die kryptografische White-Box selbst wurde mathematisch nicht gebrochen.** Kein roher AES- oder RSA-Schlüssel lag jemals im Arbeitsspeicher, kein Hardware-AES-Befehl wurde aufgerufen, eine Seitenkanalanalyse (DCA) lieferte bis zur dritten Ordnung kein verwertbares Signal. Beide Schlüsselzweige erwiesen sich in §17 bis §19 als widerstandsfähig gegen die untersuchten Fault- und Live-Extraktionsangriffe. Und trotzdem wurde das Schutzziel der App-Exklusivität vollständig umgangen, weil der Client den eingebetteten Schlüssel überhaupt nie kennen musste.

Die zentrale Erkenntnis lautet: Eine White-Box, die sich offline als Orakel aufrufen lässt, schützt zwar die *Vertraulichkeit* eines Schlüssels, aber nicht die *Fähigkeit, mit ihm zu rechnen*. Man muss den Schlüssel nicht aus dem Tresor stehlen, wenn man den gesamten Tresor in einen rund ein Megabyte kleinen Python-Snapshot packen und als Dienst aufrufen kann. Möglich wurde diese Umgehung durch drei Designentscheidungen der Gesamtarchitektur:

1. **Global-statisches Schlüsselmaterial:** Die eingebetteten White-Box-Objekte (`data1` und `data2`) sind in jeder App-Installation weltweit identisch und rotieren nicht einmal bei App-Updates (§15).
2. **Hohe Portabilität der White-Box:** Die Bibliothek ist weder an den Prozess noch an Hardware-Merkmale des Telefons gebunden, initialisiert sich im Interpreter selbst und händigt nach zwei Blockaufrufen einen gewöhnlichen Standard-AES-256-Schlüssel (`SK2`) aus (§10 und §11).
3. **Fehlende serverseitige Geräte-Attestierung:** Der Server verlässt sich beim Registrierungs-Handshake allein auf die mit dem globalen `data1` gebildete Signatur, verlangt aber keinen hardware-gestützten Nachweis (etwa über Android Play Integrity / Hardware Key Attestation), dass die Anfrage von einer unveränderten Original-App auf einem echten Smartphone stammt.

Ausdrücklicher Geltungsbereich (was NICHT betroffen ist): Belegt wurde ausschließlich der Zugriff auf **eigene Daten mit eigenen Zugangsdaten** über einen alternativen Client. Ausdrücklich intakt und an keiner Stelle beeinträchtigt sind:

- die **Nutzer-Authentifizierung** (ohne gültige E-Mail-Adresse und Passwort des Kontoinhabers ist kein Login möglich),
- die **Zugriffskontrolle zwischen Konten** (ein Zugriff auf fremde Patientendaten ist ausgeschlossen) sowie
- die **Transport-Sicherheit (TLS)** und der **lokale Hardware-Keystore (Titan-M, §2)**.

Durchbrochen ist einzig die Annahme der **Client-Exklusivität**, also die Vorstellung, nur die offizielle App könne das ALE-Protokoll sprechen. Die wirksame Gegenmaßnahme für Hersteller lautet daher nicht „eine noch komplexere White-Box bauen“, sondern die Vertrauensgrenze auf den Server und die Hardware zu verlagern: Durch eine serverseitig geprüfte, hardware-attestierte Geräte-Echtheitsprüfung in Kombination mit individuell pro Gerät und Konto provisionierten Schlüsseln statt einer globalen Konstante. Damit wäre jedes aus einer generischen APK-Datei extrahierte Binary für Dritte wertlos, selbst wenn es vollständig reversiert würde.

21. Ethik und Geltungsbereich

Sämtliche für diesen Bericht durchgeführten Analysen fanden ausschließlich mit eigenen Zugangsdaten, an eigenen Gesundheitsdaten und auf eigenen, ungerooteten Geräten statt. Es wurde kein Schlüsselmaterial, kein Snapshot, kein Klartext und kein personenbezogenes Datum veröffentlicht oder weitergegeben. Die Lösung dient ausdrücklich keinem medizinischen Therapie-Einsatz.

Dieser Bericht dokumentiert den technischen Weg zu einer persönlichen Interoperabilitäts-Lösung und fasst die dabei gewonnenen Erkenntnisse über moderne App-Härtung, White-Box-Kryptografie und den Bau eines `.so`-freien Clients zusammen. Er ist weder ein Aufruf noch eine Anleitung zum Angriff auf fremde Konten oder Systeme, sondern dient dem Fachwissen in Reverse Engineering, Kryptografie und IT-Sicherheit.

Glossar

- **ALE** — Application-Level Encryption: Zusätzliche Verschlüsselung der API-Nutzlast innerhalb der App, oberhalb von TLS.
- **RASP** — Runtime Application Self-Protection: Native Schutzschicht, die Manipulation, Debugger und Root erkennt und die App gezielt abstürzen lässt.
- **DexGuard** — kommerzieller Obfuskator/Packer für Android, der unter anderem native Bibliotheken verschlüsselt ausliefert.
- **White-Box-Kryptografie** — Implementierung, die den Schlüssel so in Lookup-Tabellen einbettet, dass er auch bei voller Code-Einsicht nicht im Klartext erscheint.
- **GCM** — Galois/Counter Mode: AES-Betriebsart, die Vertraulichkeit (über CTR) und Integrität (über einen Auth-Tag) zugleich liefert.
- **CTR** — Counter Mode: Betriebsart, die den Schlüsselstrom durch Verschlüsseln aufsteigender Zählerblöcke erzeugt.
- **PSS** — Probabilistic Signature Scheme: Randomisiertes RSA-Signaturverfahren.
- **CRT** — Chinesischer Restsatz: Beschleunigung der RSA-Berechnung über die beiden Primfaktoren.
- **Montgomery-Multiplikation** — Verfahren für schnelle modulare Multiplikation großer Zahlen, das die teure Division bei der Rest-Berechnung durch Verschiebungen ersetzt. Es rechnet in einer „Montgomery-Form“ (Faktor R), die $R^{-1} \bmod n$ am Ende wieder aufhebt.
- **CIOS** — Coarsely Integrated Operand Scanning: Die verbreitete, schleifenverschränkte Standardvariante der Montgomery-Multiplikation (Multiplikation und Reduktion ineinander verzahnt).
- **RNS** — Restklassensystem (Residue Number System): Stellt eine große Zahl durch ihre Reste modulo mehrerer kleiner, teilerfremder Moduln dar, sodass Multiplikationen spurweise parallel laufen. Das Signierwerk nutzt eine redundante Limb-Variante dieses Prinzips.
- **Fenster-Exponentiation** — Potenzierung m^d, die den Exponenten in Blöcken („Fenstern“) abarbeitet und dabei eine vorberechnete Kleinpotenz-Tabelle nutzt, schneller als Bit für Bit.
- **DFA** — Differential Fault Analysis: Schlüsselrückgewinnung durch gezielt eingestreute Rechenfehler.
- **DCA** — Differential Computation Analysis: Seitenkanalangriff auf White-Boxes über aufgezeichnete Rechenspuren.
- **NEON / SIMD** — Vektor-Befehlssatz von ARM (Single Instruction, Multiple Data).
- **ABI** — Application Binary Interface: Aufrufkonvention (Register- und Stack-Belegung) einer Funktion.

- **ELF** — Executable and Linkable Format: Ausführbares Binärformat unter Linux und Android.
- **Smali** — lesbare Disassembly-Form des Android-Bytecodes (Dalvik).
- **Leaf** — kleinste Tabellen-Primitive, aus denen die White-Box-Blockfunktion zusammengesetzt ist.
- **Hook / Stub** — ein Eingriffspunkt, an dem der Interpreter eine native Routine abfängt und durch eine gleichwertige Python-Funktion ersetzt (Hook). Die ersetzende Funktion selbst heißt Stub.
- **Snapshot** — eingefrorener, vorbereiteter Interpreter-Speicher, der die native `.so`-Datei zur Laufzeit ersetzt.
- **SK2** — der 32-Byte-Sitzungsschlüssel, mit dem alle Glukose-Antworten entschlüsselt werden.

Referenzen

- [1] IamDiesel: *LibreLinkUp Application-Level-Encryption — ein Reverse-Engineering-Writeup* (Teil 1, öffentliche Fassung, 2026). https://github.com/IamDiesel/librelinkup-ale-writeup
- [2] Chow, S., Eisen, P., Johnson, H., van Oorschot, P. C.: *White-Box Cryptography and an AES Implementation.* In: Nyberg, K., Heys, H. M. (Hrsg.): Selected Areas in Cryptography — SAC 2002. Lecture Notes in Computer Science, Bd. 2595, S. 250–270. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-36492-7_17
- [3] Billet, O., Gilbert, H., Ech-Chatbi, C.: *Cryptanalysis of a White Box AES Implementation.* In: Handschuh, H., Hasan, M. A. (Hrsg.): Selected Areas in Cryptography — SAC 2004. Lecture Notes in Computer Science, Bd. 3357, S. 227–240. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30564-4_16
- [4] Piret, G., Quisquater, J.-J.: *A Differential Fault Attack Technique against SPN Structures, with Application to the AES and KHAZAD.* In: Walter, C. D., Koç, Ç. K., Paar, C. (Hrsg.): Cryptographic Hardware and Embedded Systems — CHES 2003. Lecture Notes in Computer Science, Bd. 2779, S. 77–88. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45238-6_7
- [5] Boneh, D., DeMillo, R. A., Lipton, R. J.: *On the Importance of Checking Cryptographic Protocols for Faults.* In: Fumy, W. (Hrsg.): Advances in Cryptology — EUROCRYPT 1997. Lecture Notes in Computer Science, Bd. 1233, S. 37–51. Springer, Heidelberg (1997). https://doi.org/10.1007/3-540-69053-0_4
- [6] National Institute of Standards and Technology: *Advanced Encryption Standard (AES).* FIPS PUB 197 (2001). https://doi.org/10.6028/NIST.FIPS.197
- [7] Dworkin, M.: *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.* NIST Special Publication 800-38D (2007). https://doi.org/10.6028/NIST.SP.800-38D
- [8] Montgomery, P. L.: *Modular Multiplication Without Trial Division.* Mathematics of Computation 44(170), S. 519–521 (1985). https://doi.org/10.1090/S0025-5718-1985-0777282-X
- [9] Koç, Ç. K., Acar, T., Kaliski, B. S.: *Analyzing and Comparing Montgomery Multiplication Algorithms.* IEEE Micro 16(3), S. 26–33 (1996). https://doi.org/10.1109/40.502403
- [10] Moriarty, K., Kaliski, B., Jonsson, J., Rusch, A.: *PKCS #1: RSA Cryptography Specifications Version 2.2.* RFC 8017, IETF (2016). https://doi.org/10.17487/RFC8017
- [11] Bos, J. W., Hubain, C., Michiels, W., Teuwen, P.: *Differential Computation Analysis: Hiding Your White-Box Designs is Not Enough.* In: Cryptographic Hardware and Embedded Systems — CHES 2016. Lecture Notes in Computer Science, Bd. 9813, S. 215–236. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53140-2_11
- [12] Wiener, M. J.: *Cryptanalysis of Short RSA Secret Exponents.* IEEE Transactions on Information Theory 36(3), S. 553–558 (1990). https://doi.org/10.1109/18.54902